

Image and Video Generations

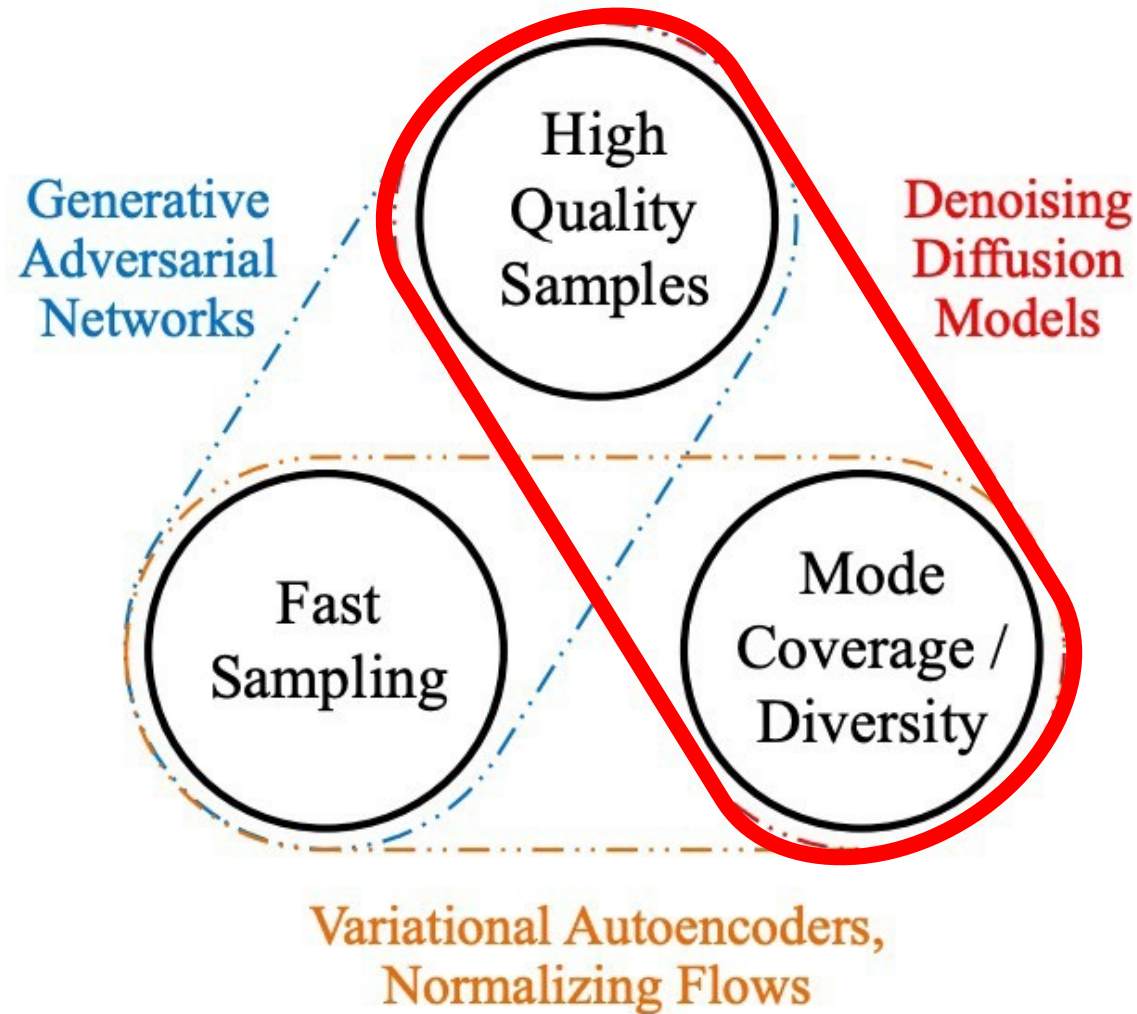
Lecture 10: Flow Matching

劉育綸

Yu-Lun (Alex) Liu



Generative Models – Comparison



Probability Flow ODE

1. Both the forward and reverse diffusion processes can be described by an **SDE** in the continuous domain.
2. A diffusion process SDE has a corresponding **ODE** that has the same marginal distribution at each time step.
3. Hence, the reverse diffusion (generation) process can be conducted by solving the corresponding ODE.

Fokker–Planck Equation

Fokker-Planck equation 描述了給定 SDE 下，機率密度 $p_t(\mathbf{x})$ 如何隨時間改變

SDE 描述個別軌跡：
$$d\mathbf{x}_t = f(\mathbf{x}_t, t)dt + g(\mathbf{x}_t, t)d\mathbf{w}$$

Fokker-Planck 描述群體分布：

$$\Rightarrow \frac{\partial p_t(\mathbf{x})}{\partial t} = -\nabla_{\mathbf{x}_t} \cdot [f(\mathbf{x}_t, t)p_t(\mathbf{x}_t)] + \frac{1}{2} \nabla_{\mathbf{x}_t}^2 [g(\mathbf{x}_t, t)g(\mathbf{x}_t, t)^T p_t(\mathbf{x}_t)]$$

但我們可以找到一個 ODE（沒有隨機項）在所有時間點產生完全相同的 $p_t(\mathbf{x})$
Fokker-Planck equation 是連接 SDE 與 ODE 的橋樑，它告訴我們如何設計 **ODE 的 deterministic flow field**，使其產生與 SDE 相同的 **marginal probability distribution**

DPM-Solver

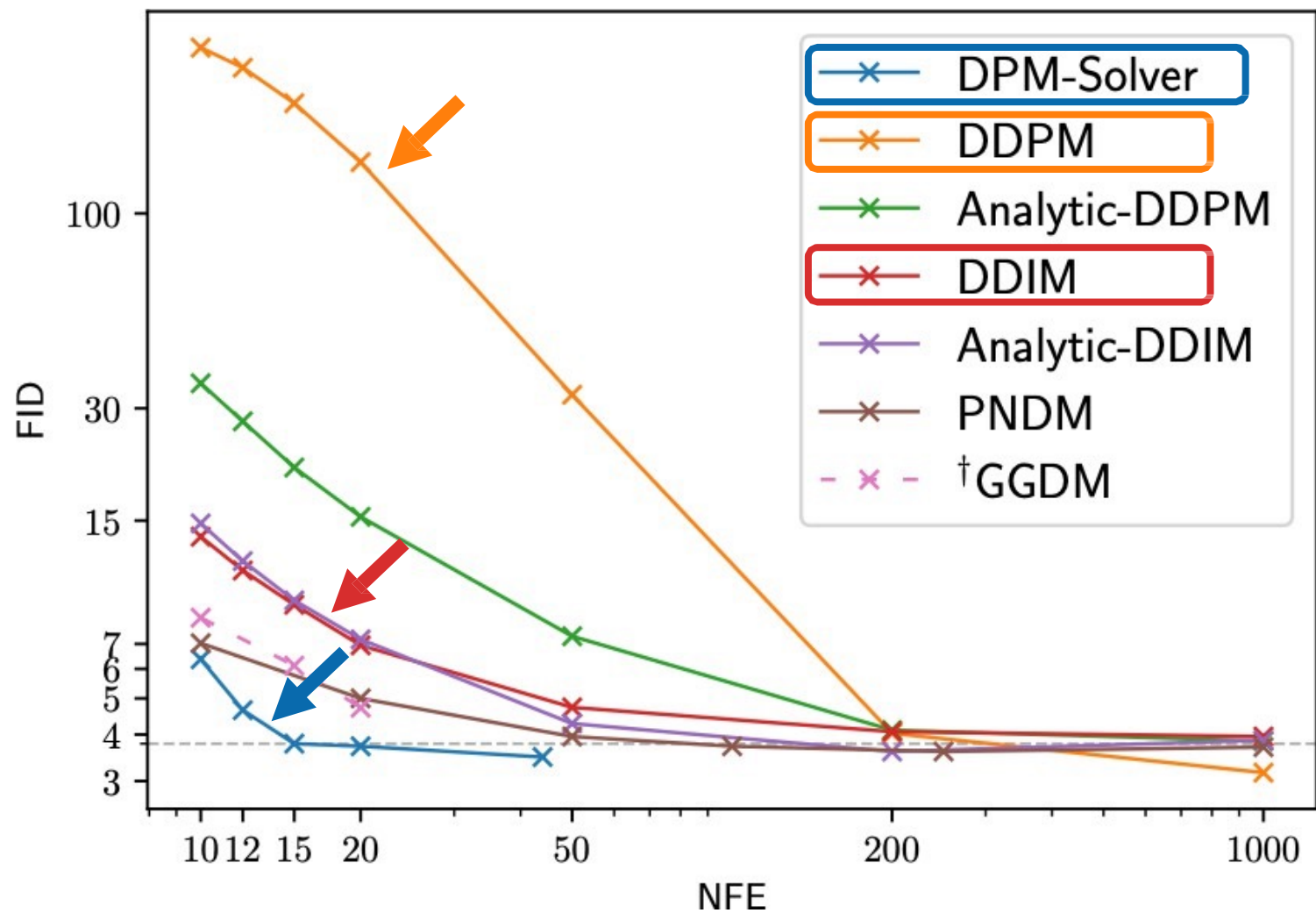
雖然我們已經將 diffusion model 從 SDE 轉換成 ODE，但使用通用 numerical solver（如 Euler、Runge-Kutta）需要 100-1000 步

Derive a **dedicated** solver for the following diffusion model ODE that can solve it accurately in just a **few steps**.

$$\frac{dx(t)}{dt} = \left(\frac{d \log \alpha_t}{dt} \right) x(t) - \sigma_t \frac{d\lambda_t}{dt} \hat{\epsilon}_\theta(x(t), t)$$

DPM-Solver 被設計來針對 diffusion ODE 的特定結構 optimize，大幅減少 number of function evaluations，在少量 steps 下仍能生成高品質影像

DPM-Solver



Flow-Based Models

- A class of generative models that aim to directly model the probability distribution of data by transforming a base distribution into a complex target distribution using a **series of invertible transformations**.
- The transformations are often parameterized by an **ODE**.
- Flow-based models are **studied earlier** than diffusion models in deep generative modeling.

Flow-Based Models

- Rezende and Mohamed, Variational Inference with Normalizing Flows, ICML 2015.
- Dinh et al., NICE: Non-linear Independent Components Estimation, ICLR 2015.
- Dinh et al., Density estimation using Real NVP, ICLR 2017.
- Grathwohl et al., FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models, ICLR 2019.
- Ho et al., Flow++: Improving Flow-Based Generative Models with Variational Dequantization and Architecture Design, ICML 2019.
- Chen et al., Residual Flows for Invertible Generative Modeling, NeurIPS 2019.
- Behrmann et al., Invertible Residual Networks, ICML 2019.
- Lipman et al., Flow Matching for Generative Modeling, ICLR 2023.
- ...

Flow-Based Models

AN INTRODUCTION TO FLOW MATCHING

diffusion model

generative modelling

normalising flows

By [Tor Fjelde](#), [Emile Mathieu](#), [Vincent Dutordoir](#)

TABLE OF CONTENTS

1. [Introduction](#)
2. [Normalising Flows](#)
 1. [Learning flow parameters by maximum likelihood](#)
 2. [Residual flow](#)
 3. [Continuous time limit](#)
3. [Flow matching](#)
 1. [Conditional Flows](#)
 2. [Gaussian probability paths](#)
 3. [But is CFM really all rainbows and unicorns?](#)
 4. [Coupling](#)
4. [Quick Summary](#)
5. [Citation](#)
6. [Acknowledgments](#)
7. [References](#)

Flow-Based Models

AN INTRODUCTION TO FLOW MATCHING

diffusion model

generative modelling

normalising flows

By [Tor Fjelde](#), [Emile Mathieu](#), [Vincent Dutordoir](#)

TABLE OF CONTENTS

1. [Introduction](#)
2. [Normalising Flows](#)
 1. [Learning flow parameters by maximum likelihood](#)
 2. [Residual flow](#)
 3. [Continuous time limit](#)
3. [Flow matching](#)
 1. [Conditional Flows](#)
 2. [Gaussian probability paths](#)
 3. [But is CFM really all rainbows and unicorns?](#)
 4. [Coupling](#)
4. [Quick Summary](#)
5. [Citation](#)
6. [Acknowledgments](#)
7. [References](#)

Notations

Diffusion Models [Generation: $T \rightarrow 0$]

- x_0 : A sample from the data distribution.
- x_T : A sample from the reference (base) distribution.

Flow Matching [Generation: $0 \rightarrow 1$]

- x_1 : A sample from the data distribution.
- x_0 : A sample from the reference (base) distribution.

Normalizing Flow

Rezende and Mohamed, Variational Inference with Normalizing Flows, ICML 2015.

Normalizing Flow

- Let ϕ be a **continuous differentiable** function, whose inverse ϕ^{-1} also exists and continuously differentiable.
- $p_0(\mathbf{x}_0)$: Reference distribution.
- $p_1(\mathbf{x}_1)$: A distribution induced by the following sampling procedure:

$$\mathbf{x}_0 \sim p_0$$

$$\mathbf{x}_1 = \phi(\mathbf{x}_0)$$

Change-of-Variable

- The **probability mass** in the original space should equal the probability mass in the transformed space:

$$p_1(\mathbf{x}_1)d\mathbf{x}_1 = p_0(\mathbf{x}_0)d\mathbf{x}_0$$

當我們將變數從 $\mathbf{x}_0$ 轉換到 $\mathbf{x}_1$ 時，總機率必須保持不變。因為他們是同一個事件，只是用不同座標系統描述

- Given $\mathbf{x}_0 = \phi^{-1}(\mathbf{x}_1)$, the **absolute value of the Jacobian determinant**

也就是說我們可以從 $\mathbf{x}_1$ 反推回 $\mathbf{x}_0$

$$d\mathbf{x}_0 = \left| \det \left(\frac{\partial \phi^{-1}(\mathbf{x}_1)}{\partial \mathbf{x}_1} \right) \right| d\mathbf{x}_1$$

measures the **local scaling** of the volume during the transformation.

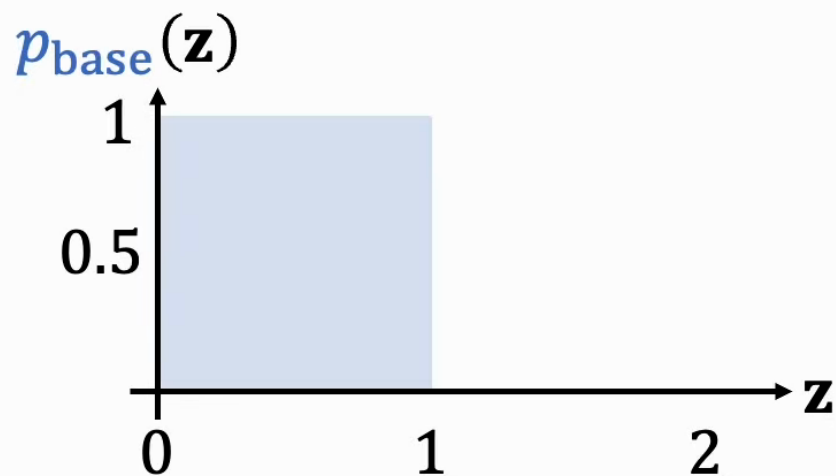
這個 **Jacobian determinant** 的絕對值測量了轉換過程中的**局部體積縮放**

為什麼 determinant 測量體積縮放？ 一維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z})$$

$$\mathbf{x} = G(\mathbf{z})$$

$$p(\mathbf{x}) = ?$$

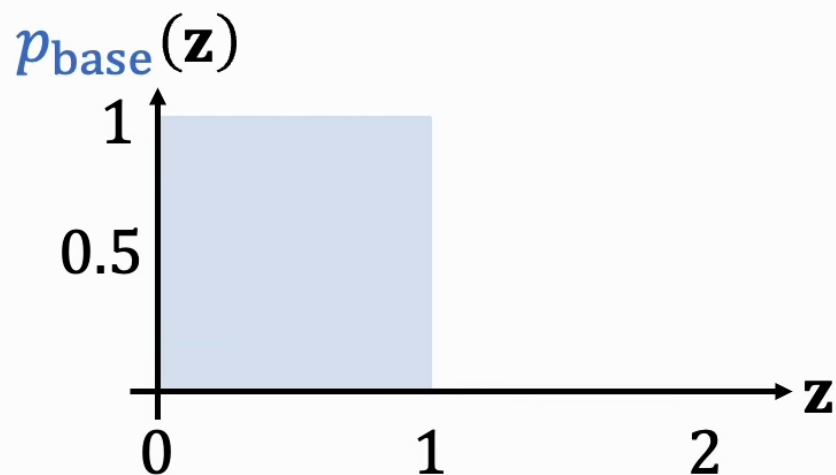


為什麼 determinant 測量體積縮放？ 一維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z})$$

$$\mathbf{x} = G(\mathbf{z})$$

$$p(\mathbf{x}) = ?$$



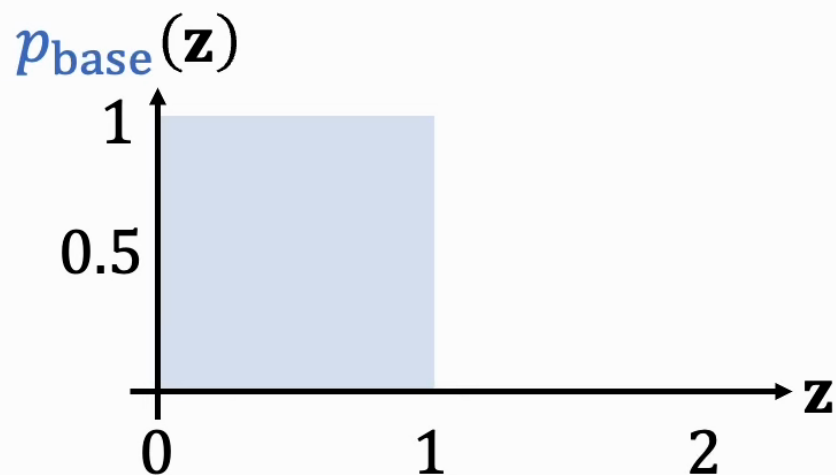
$$\mathbf{x} = 2\mathbf{z}$$

為什麼 determinant 測量體積縮放？ 一維例子

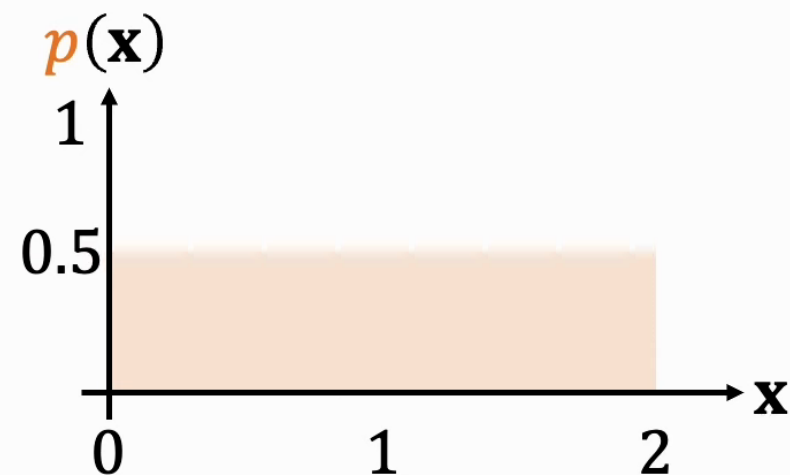
$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z})$$

$$\mathbf{x} = G(\mathbf{z})$$

$$p(\mathbf{x}) = ?$$



$$\mathbf{x} = 2\mathbf{z}$$

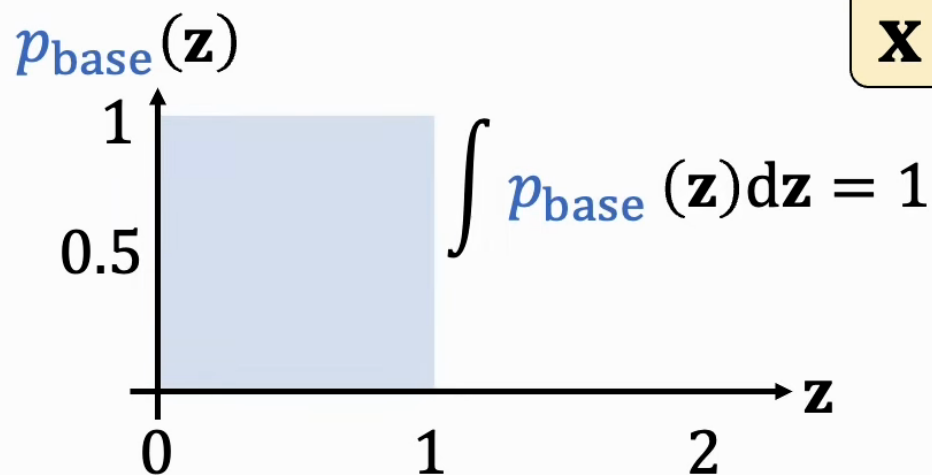


為什麼 determinant 測量體積縮放？ 一維例子

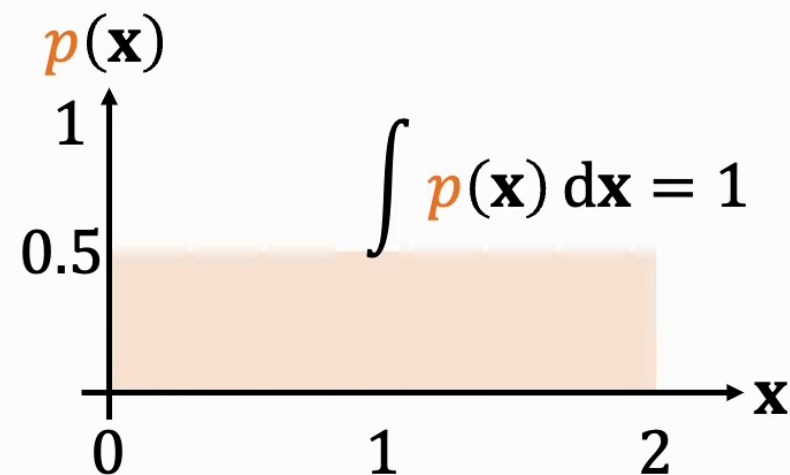
$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z})$$

$$\mathbf{x} = G(\mathbf{z})$$

$$p(\mathbf{x}) = ?$$



$$\mathbf{x} = 2\mathbf{z}$$



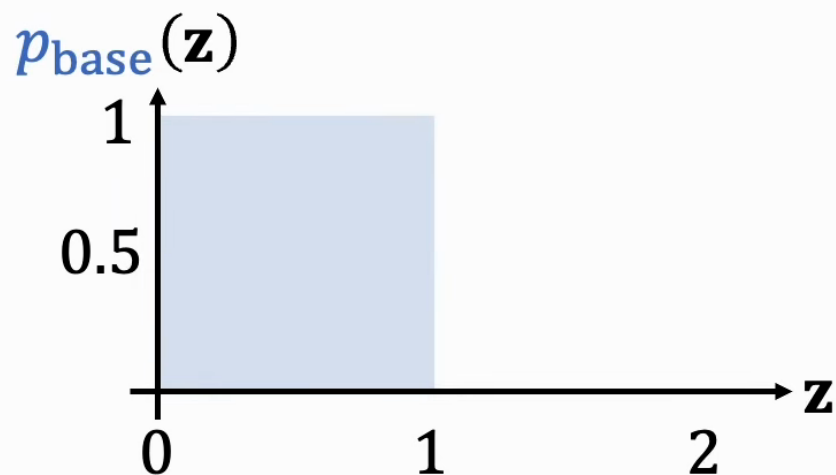
為什麼 determinant 測量體積縮放？ 一維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z})$$

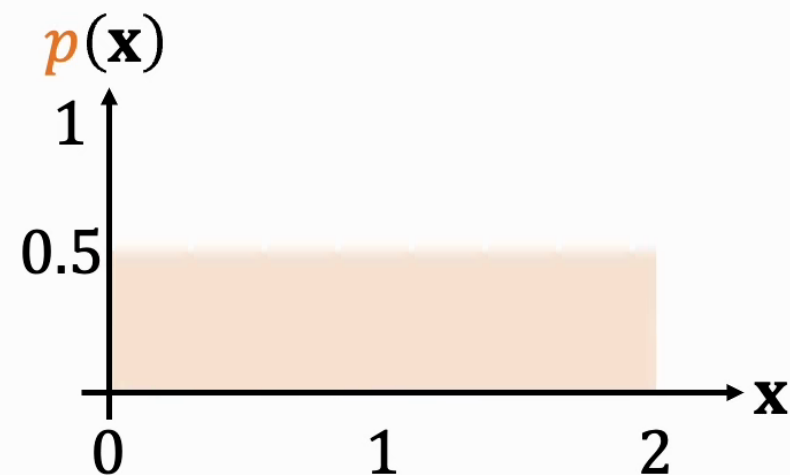
$$\mathbf{x} = G(\mathbf{z})$$

$$p(\mathbf{x}) = \frac{1}{2} p_{\text{base}}(\mathbf{z})$$

$$\mathbf{z} = G^{-1}(\mathbf{x})$$



$$\mathbf{x} = 2\mathbf{z}$$

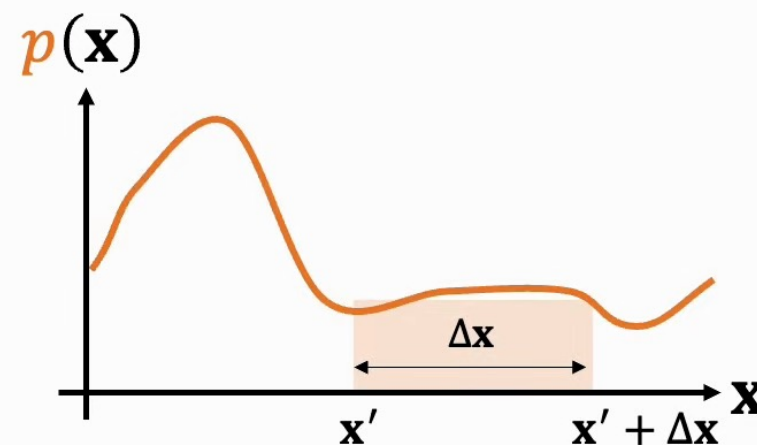
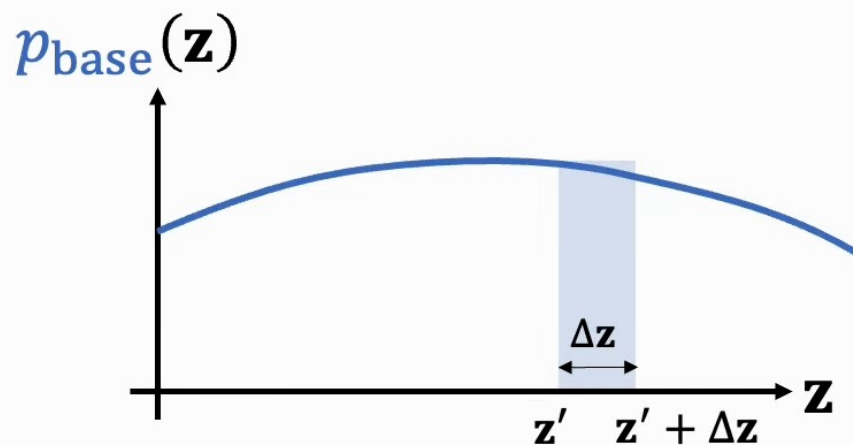


為什麼 determinant 測量體積縮放？ 一維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z})$$

$$\mathbf{x} = G(\mathbf{z})$$

$$p(\mathbf{x}')\Delta\mathbf{x} = p_{\text{base}}(\mathbf{z}')\Delta\mathbf{z}$$

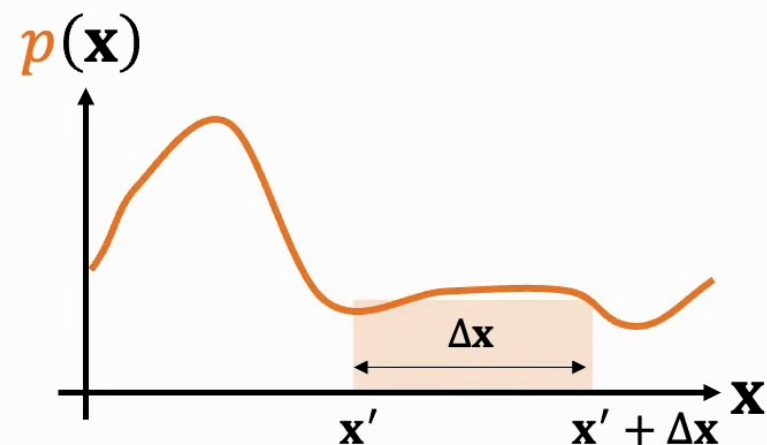
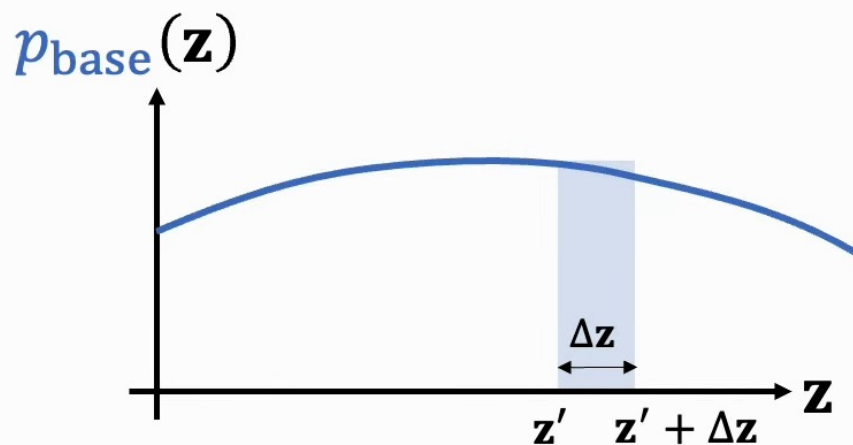


為什麼 determinant 測量體積縮放？ 一維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z})$$

$$\mathbf{x} = G(\mathbf{z})$$

$$p(\mathbf{x}') = p_{\text{base}}(\mathbf{z}') \frac{\Delta \mathbf{z}}{\Delta \mathbf{x}}$$

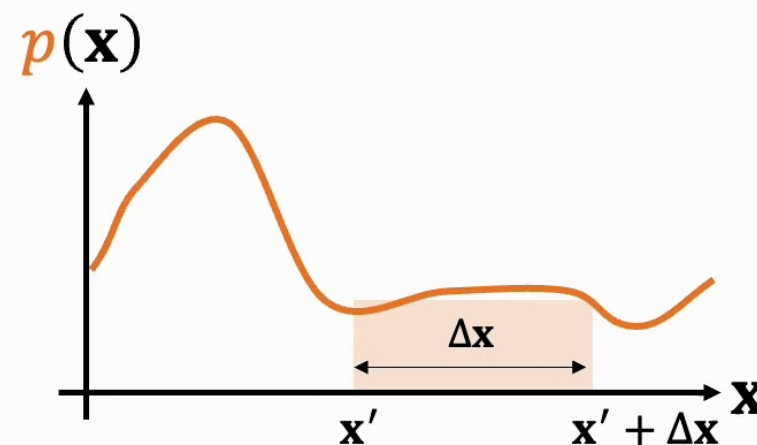
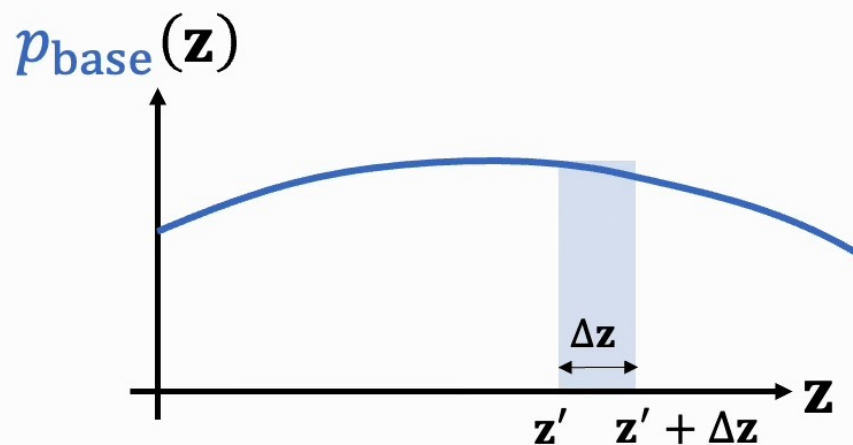


為什麼 determinant 測量體積縮放？ 一維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z})$$

$$\mathbf{x} = G(\mathbf{z})$$

$$p(\mathbf{x}') = p_{\text{base}}(\mathbf{z}') \frac{\partial \mathbf{z}}{\partial \mathbf{x}}$$

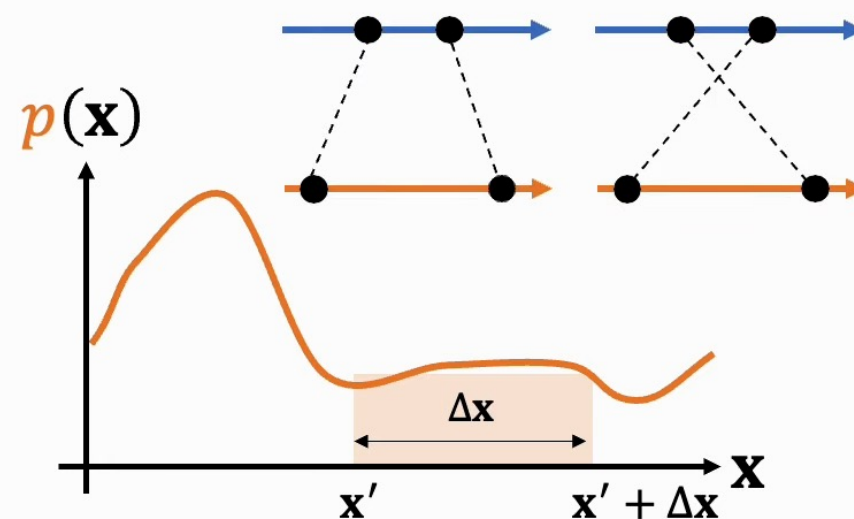
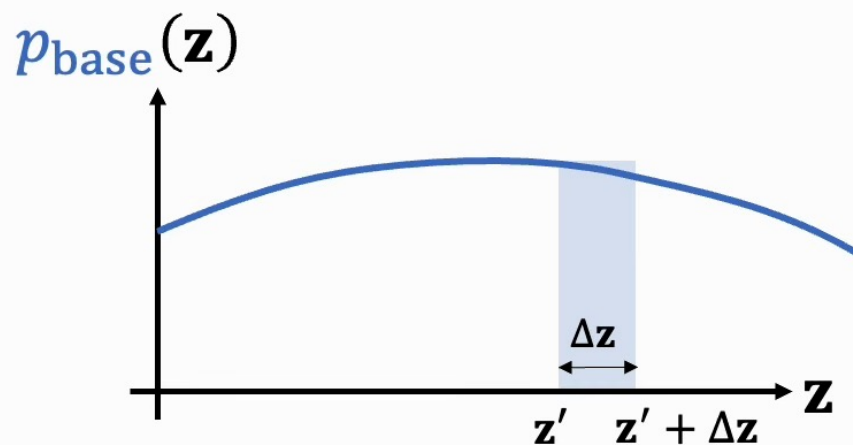


為什麼 determinant 測量體積縮放？ 一維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z})$$

$$\mathbf{x} = G(\mathbf{z})$$

$$p(\mathbf{x}') = p_{\text{base}}(\mathbf{z}') \left| \frac{\partial \mathbf{z}}{\partial \mathbf{x}} \right|$$

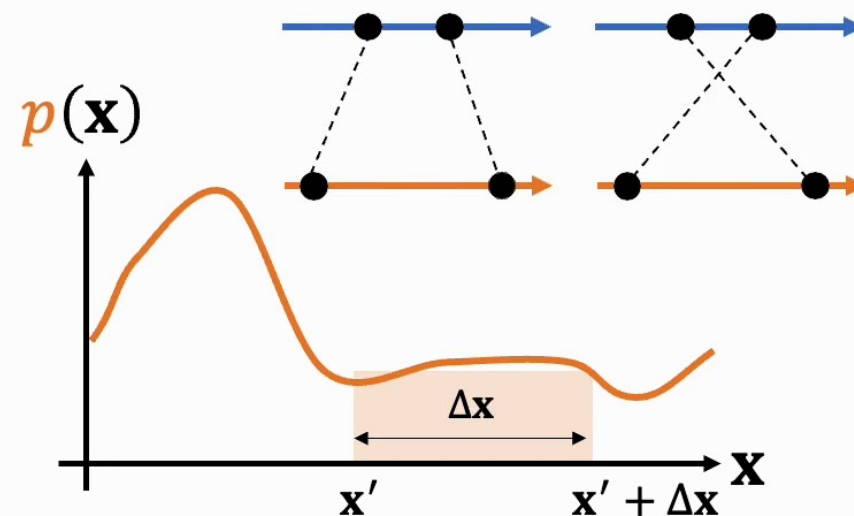
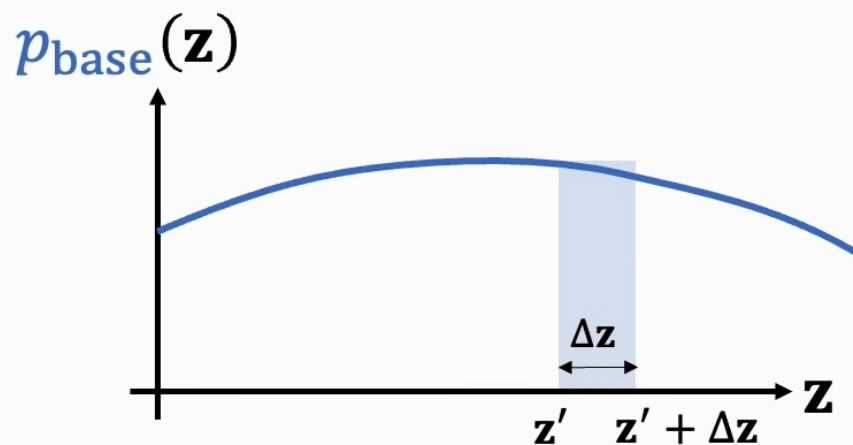


為什麼 determinant 測量體積縮放？ 一維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z})$$

$$\mathbf{x} = G(\mathbf{z})$$

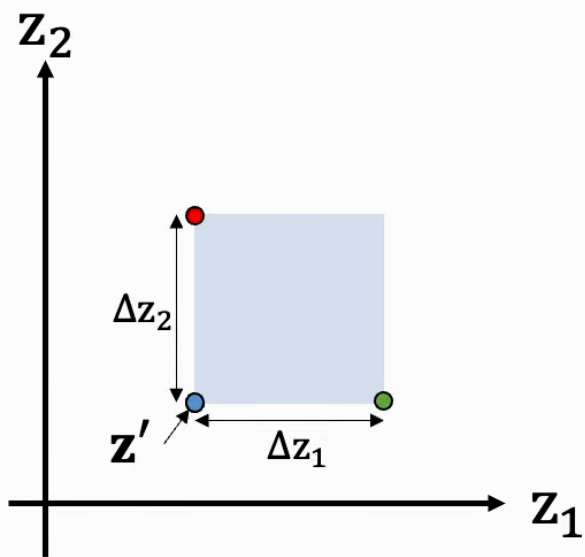
$$p(\mathbf{x}') = p_{\text{base}}(\mathbf{z}') \left| \frac{\partial \mathbf{z}}{\partial \mathbf{x}} \right|$$



為什麼 determinant 測量體積縮放？

二維例子

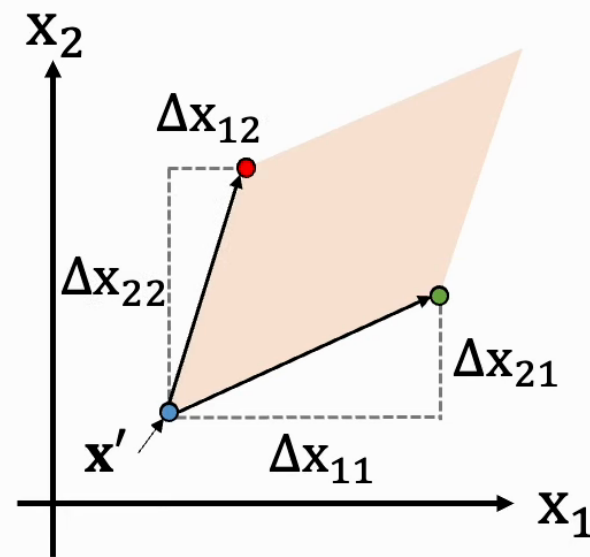
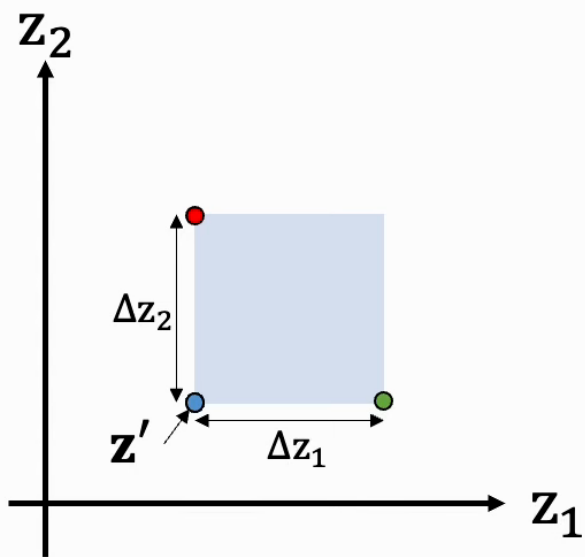
$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z}) \quad \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = G \left(\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \right)$$



為什麼 determinant 測量體積縮放？

二維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z}) \quad \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = G \left(\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \right)$$

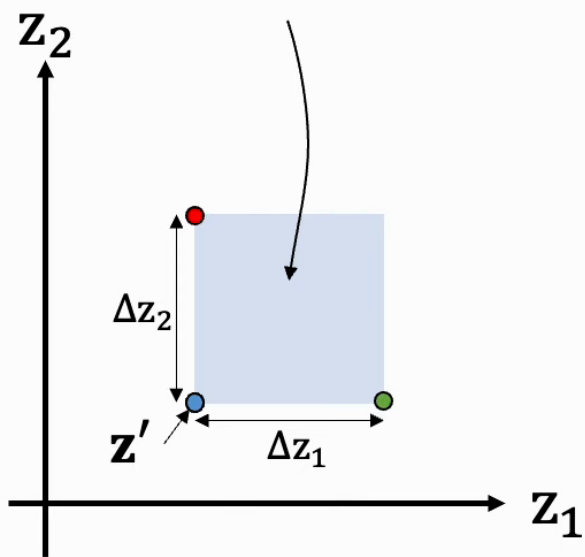


為什麼 determinant 測量體積縮放？

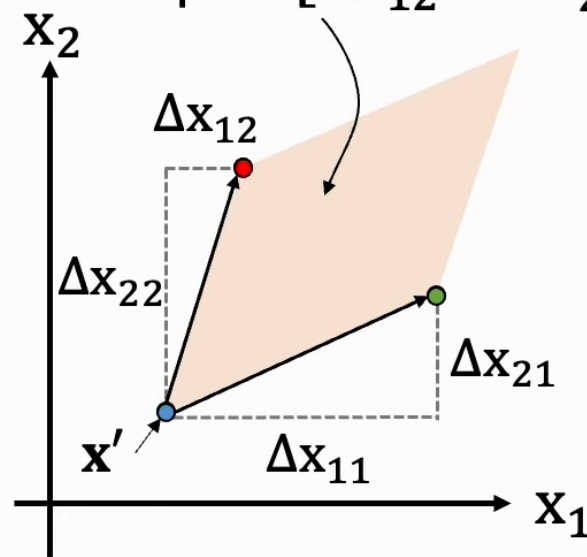
二維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z}) \quad \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = G \left(\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \right)$$

$$\text{Area} = \Delta z_1 \Delta z_2$$



$$\text{Area} = \left| \det \begin{bmatrix} \Delta x_{11} & \Delta x_{21} \\ \Delta x_{12} & \Delta x_{22} \end{bmatrix} \right|$$



為什麼 determinant 測量體積縮放？

二維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z}) \quad \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \end{bmatrix} = G \left(\begin{bmatrix} \mathbf{z}_1 \\ \mathbf{z}_2 \end{bmatrix} \right)$$

$$p_{\text{base}}(\mathbf{z}') \Delta z_1 \Delta z_2 = p(\mathbf{x}') \left| \det \begin{bmatrix} \Delta \mathbf{x}_{11} & \Delta \mathbf{x}_{21} \\ \Delta \mathbf{x}_{12} & \Delta \mathbf{x}_{22} \end{bmatrix} \right|$$

為什麼 determinant 測量體積縮放？

二維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z}) \quad \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = G \left(\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \right)$$

$$p_{\text{base}}(\mathbf{z}') = p(\mathbf{x}') \frac{1}{\Delta z_1 \Delta z_2} \left| \det \begin{bmatrix} \Delta x_{11} & \Delta x_{21} \\ \Delta x_{12} & \Delta x_{22} \end{bmatrix} \right|$$

為什麼 determinant 測量體積縮放？

二維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z}) \quad \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = G \left(\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \right)$$

$$p_{\text{base}}(\mathbf{z}') = p(\mathbf{x}') \left| \det \begin{bmatrix} \Delta x_{11}/\Delta z_1 & \Delta x_{21}/\Delta z_1 \\ \Delta x_{12}/\Delta z_2 & \Delta x_{22}/\Delta z_2 \end{bmatrix} \right|$$

為什麼 determinant 測量體積縮放？

二維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z}) \quad \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = G \left(\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \right)$$

$$p_{\text{base}}(\mathbf{z}') = p(\mathbf{x}') \left| \det \begin{bmatrix} \frac{\partial x_1}{\partial z_1} & \frac{\partial x_2}{\partial z_1} \\ \frac{\partial x_1}{\partial z_2} & \frac{\partial x_2}{\partial z_2} \end{bmatrix} \right|$$

為什麼 determinant 測量體積縮放？

二維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z}) \quad \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = G \left(\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \right)$$

$$p_{\text{base}}(\mathbf{z}') = p(\mathbf{x}') \left| \det \begin{bmatrix} \frac{\partial x_1}{\partial z_1} & \frac{\partial x_1}{\partial z_2} \\ \frac{\partial x_2}{\partial z_1} & \frac{\partial x_2}{\partial z_2} \end{bmatrix} \right|$$

為什麼 determinant 測量體積縮放？

二維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z}) \quad \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = G \left(\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \right)$$

$$p_{\text{base}}(\mathbf{z}') = p(\mathbf{x}') \left| \det \begin{bmatrix} \frac{\partial x_1}{\partial z_1} & \frac{\partial x_1}{\partial z_2} \\ \frac{\partial x_2}{\partial z_1} & \frac{\partial x_2}{\partial z_2} \end{bmatrix} \right|$$

Jacobian matrix J_G

為什麼 determinant 測量體積縮放？

二維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z}) \quad \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = G \left(\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \right)$$

$$p(\mathbf{x}') |\det[\mathbf{J}_G]| = p_{\text{base}}(\mathbf{z}')$$

為什麼 determinant 測量體積縮放？

二維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z}) \quad \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = G \left(\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \right)$$

$$p(\mathbf{x}') = p_{\text{base}}(\mathbf{z}') \frac{1}{|\det[\mathbf{J}_G]|}$$

為什麼 determinant 測量體積縮放？

二維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z}) \quad \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = G \left(\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \right)$$

$$p(\mathbf{x}') = p_{\text{base}}(\mathbf{z}') |\det[\mathbf{J}_{G^{-1}}]|$$

為什麼 determinant 測量體積縮放？

二維例子

$$\mathbf{z} \sim p_{\text{base}}(\mathbf{z}) \quad \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = G \left(\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \right)$$

**CHANGE-OF-VARIABLE
FORMULA**

$$p(\mathbf{x}') = p_{\text{base}}(\mathbf{z}') |\det[\mathbf{J}_{G^{-1}}]|$$

Change-of-Variable

- Combining the two leads to

$$p_1(\mathbf{x}_1) = p_0(\mathbf{x}_0) \frac{d\mathbf{x}_0}{d\mathbf{x}_1} = p_0(\phi^{-1}(\mathbf{x}_1)) \left| \det \left(\frac{\partial \phi^{-1}(\mathbf{x}_1)}{\partial \mathbf{x}_1} \right) \right|$$

$$\log p_1(\mathbf{x}_1) = \log p_0(\phi^{-1}(\mathbf{x}_1)) + \log \left| \det \left(\frac{\partial \phi^{-1}(\mathbf{x}_1)}{\partial \mathbf{x}_1} \right) \right|$$

- Let's simply denote the **push-forward** operator as $[\phi]_*$:

$$p_1(\mathbf{x}_1) = p_0(\phi^{-1}(\mathbf{x}_1)) \left| \det \left(\frac{\partial \phi^{-1}(\mathbf{x}_1)}{\partial \mathbf{x}_1} \right) \right| = [\phi]_* p_0(\mathbf{x}_1)$$

Quiz

- $p_0(\mathbf{x}_0) = \mathcal{N}(\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(\mathbf{x}_0 - \mu)^2}{2\sigma^2}}$
- $\phi(\mathbf{x}_0) = a\mathbf{x}_0 + b \quad (a \neq 0)$
- $\phi^{-1}(\mathbf{x}_1) = \frac{\mathbf{x}_1 - b}{a}$

Q. What is $p_1(\mathbf{x}_1) = [\phi]_* p_0(\mathbf{x}_1)$?

Answer

$$\begin{aligned} [\phi]_* p_0(\mathbf{x}_1) &= p_0\left(\frac{\mathbf{x}_1 - b}{a}\right) \frac{1}{a} \\ &= \frac{1}{a} \cdot \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{\left(\left(\frac{\mathbf{x}_1 - b}{a}\right) - \mu\right)^2}{2\sigma^2}} \\ &= \frac{1}{\sqrt{2\pi(a\sigma)^2}} e^{-\frac{(\mathbf{x}_1 - (a\mu + b))^2}{2(a\sigma)^2}} \\ &= \mathcal{N}(a\mu + b, a^2\sigma^2) \end{aligned}$$

The result is consistent with the linearity of Gaussians!

Normalizing Flow

Given the samples $\mathbf{x}_1$ from the unknown data distribution p_1 , we can consider learning the invertible function $\phi_\theta(\mathbf{x}_0)$ by minimizing the following objective function:

$$\mathcal{L} = -\mathbb{E}_{\mathbf{x}_1 \sim p_1} \log([\phi]_* p_0(\mathbf{x}_1))$$

Normalizing Flow

Challenges

- How to learn an **invertible** function and compute the inverse?

Neural networks 通常不可逆（如 ReLU 會丟失信息），保證可逆性需要特殊架構（如 coupling layers），但會限制 model 表達能力

- How to compute the **Jacobian**?

計算 Jacobian 昂貴，對 d 維 data，Jacobian 是 $d \times d$ 矩陣，計算 determinant 需要 $O(d^3)$ 複雜度，在高維度時計算幾乎變得不可行

Residual Flow

將複雜 mapping 分解成 T 個簡單 mapping 的組合

Represent the function ϕ as a **composition** of multiple functions:

$$\phi = \phi_{(T-1) \cdot \Delta t} \circ \cdots \circ \phi_{1 \cdot \Delta t} \circ \phi_{0 \cdot \Delta t}$$

where each function is defined as:

$$\phi_t(\mathbf{x}_t) = \mathbf{x}_t + \Delta t \cdot u_t(\mathbf{x}_t).$$

Residual
update

and $\Delta t = \frac{1}{T}$.

- 當 Δt 足夠小時，invertible 變得容易
- 當 Δt 足夠小時，Jacobian 計算高效，只需計算 trace， $O(d^3) \rightarrow O(d)$
- 多個簡單變換組合 = 複雜變換，類似 deep networks 或 ResNets，簡單的 layers 堆疊就能產生複雜的函數

Continuous Normalizing Flow

What if $T \rightarrow \infty \Rightarrow \Delta t \rightarrow 0$?

$$\lim_{\Delta t \rightarrow 0} \frac{\phi_t(\mathbf{x}_t) - \mathbf{x}_t}{\Delta t} = \lim_{\Delta t \rightarrow 0} \frac{\mathbf{x}_{t+\Delta t} - \mathbf{x}_t}{\Delta t} = \frac{d\mathbf{x}_t}{dt} = u_t(\mathbf{x}_t).$$

ODE formulation!

Solve the ODE with the initial condition $\mathbf{x}_0$:

$$\mathbf{x}_t = \mathbf{x}_0 + \int_0^t u_s(\mathbf{x}_s) ds = \mathbf{x}_0 + \int_0^t u_s(\phi_s(\mathbf{x}_0)) ds$$

Quiz

Recall Fokker–Planck equation:

$$d\mathbf{x}_t = f(\mathbf{x}_t, t)dt + g(\mathbf{x}_t, t)d\mathbf{w}$$

$$\Rightarrow \frac{\partial p_t(\mathbf{x})}{\partial t} = -\nabla_{\mathbf{x}_t} \cdot [f(\mathbf{x}_t, t)p_t(\mathbf{x}_t)] + \frac{1}{2} \nabla_{\mathbf{x}_t}^2 [g(\mathbf{x}_t, t)g(\mathbf{x}_t, t)^T p_t(\mathbf{x}_t)]$$

Q. When $\frac{d\mathbf{x}_t}{dt} = u_t(\mathbf{x}_t)$, what is $\frac{\partial p_t(\mathbf{x}_t)}{\partial t}$?

Answer

A. From Fokker–Planck equation:

$$d\mathbf{x}_t = f(\mathbf{x}_t, t)dt + \cancel{g(\mathbf{x}_t, t)d\mathbf{w}}$$

$$\Rightarrow \frac{\partial p_t(\mathbf{x})}{\partial t} = -\nabla_{\mathbf{x}_t} \cdot [f(\mathbf{x}_t, t)p_t(\mathbf{x}_t)] + \frac{1}{2} \nabla_{\mathbf{x}_t}^2 [\cancel{g(\mathbf{x}_t, t)g(\mathbf{x}_t, t)^T p_t(\mathbf{x}_t)}]$$

$$\frac{d\mathbf{x}_t}{dt} = u_t(\mathbf{x}_t) \Rightarrow \frac{\partial p_t(\mathbf{x}_t)}{\partial t} = -\nabla \cdot [u_t(\mathbf{x}_t)p_t(\mathbf{x}_t)]$$

Continuous Normalizing Flow

$$\frac{\partial p_t(\mathbf{x}_t)}{\partial t} = -\nabla \cdot [u_t(\mathbf{x}_t)p_t(\mathbf{x}_t)]$$

$$\Rightarrow \dots \Rightarrow \frac{\partial \log p_t(\mathbf{x}_t)}{\partial t} = -\nabla \cdot u_t(\mathbf{x}_t)$$

$$\Rightarrow \log p_1(\mathbf{x}_1) = \log p_0(\mathbf{x}_0) - \int_0^1 \nabla \cdot u_t(\mathbf{x}_t) dt$$

Continuous Normalizing Flow

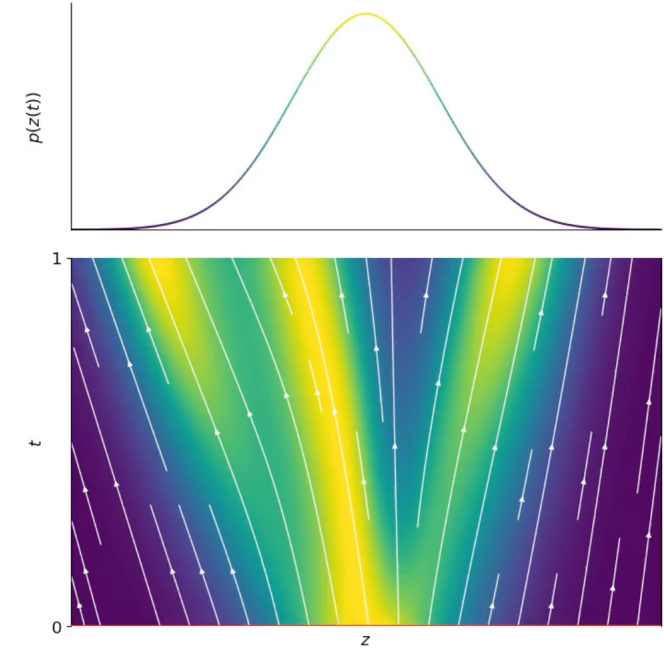
Given the samples $\mathbf{x}$ from the unknown data distribution p_1 , now we can consider learning the **vector field** $u_\theta(\mathbf{x}_t, t)$ (instead of $\phi_\theta(\mathbf{x}_0)$) by minimizing the same objective function:

$$\mathcal{L} = -\mathbb{E}_{\mathbf{x}_1 \sim p_1} \log p_1(\mathbf{x}_1)$$

There are lots details about how to compute this loss, which we will skip here.

Continuous Normalizing Flow

- 與 Diffusion 的相似性
 - 都是逐步變換
 - 都可以用 ODE 描述
 - 都利用 residual 連接
- Challenges
- The computation of the objective involves solving an ODE, which is very time-consuming.



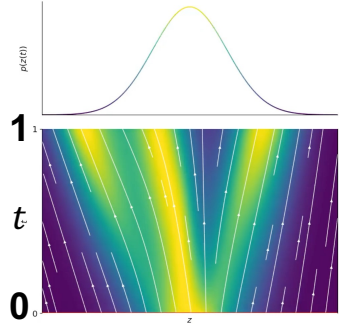
Conditional Flow Matching

Lipman et al., Flow Matching for Generative Modeling, ICLR 2023.

Flow Matching

- Let's assume we already know a **vector field** $u_t(t)$ that satisfies

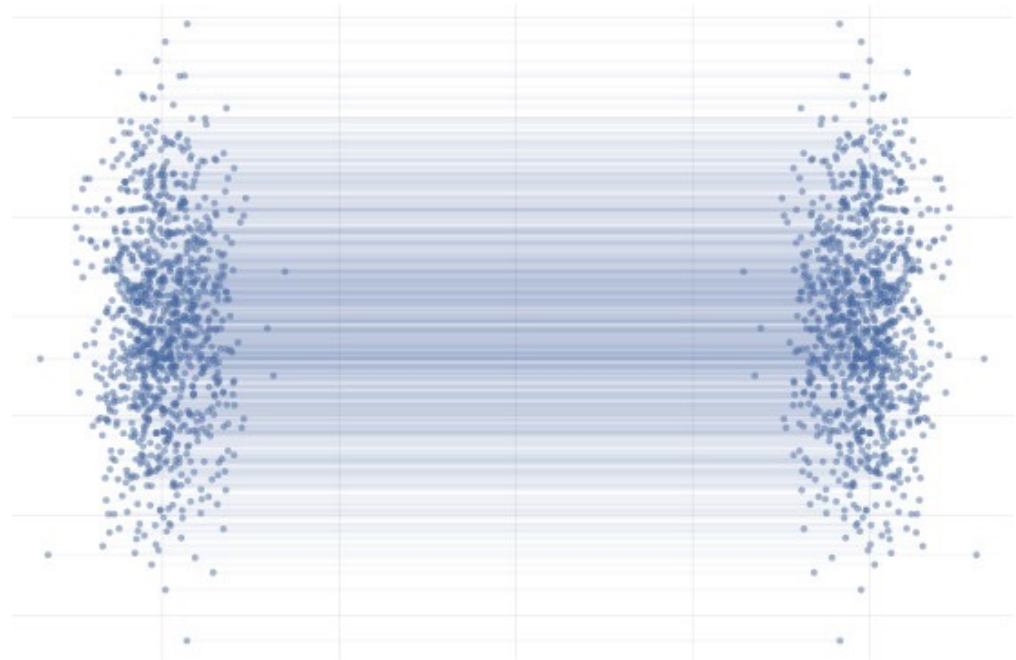
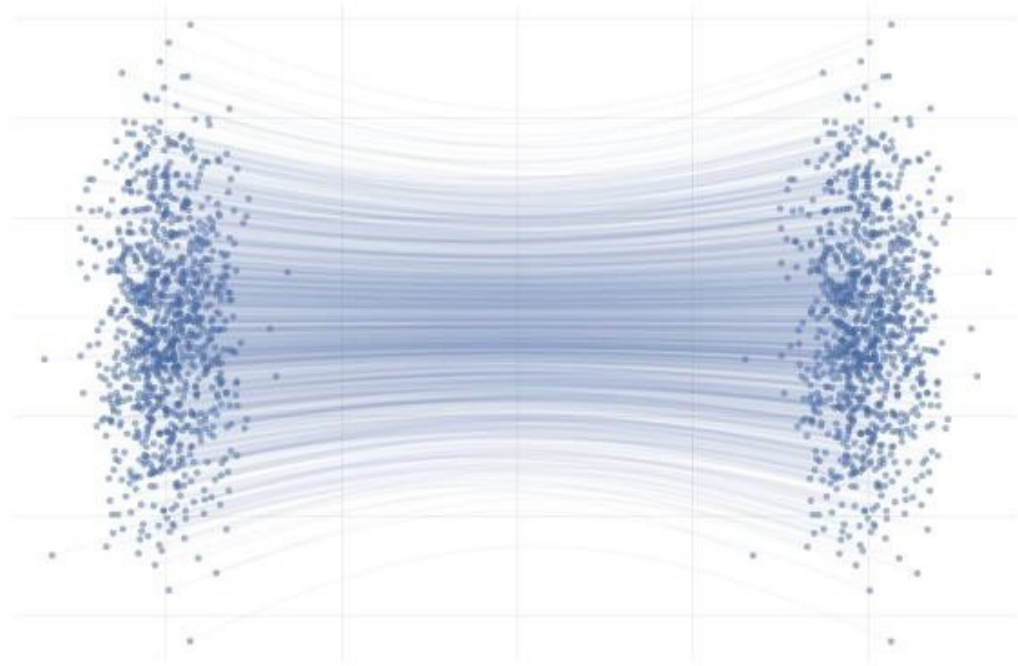
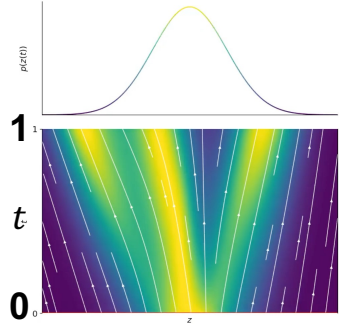
$$\log p_1(\mathbf{x}_1) = \log p_0(\mathbf{x}_0) - \int_0^1 \nabla \cdot u_t(t) dt$$



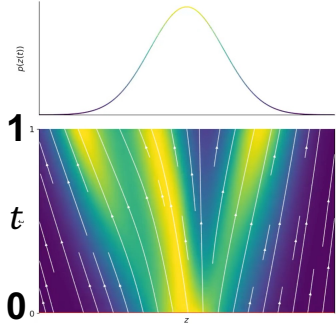
Flow Matching

Is such a vector field **unique**?

No. It is not unique.



Flow Matching



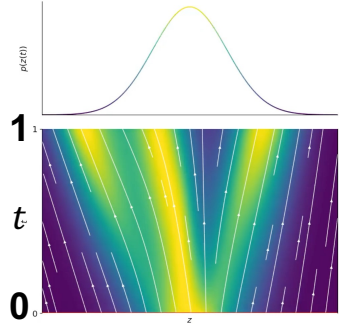
- Let's assume we already know a **vector field** $u_t(t)$ that satisfies

$$\log p_1(\mathbf{x}_1) = \log p_0(\mathbf{x}_0) - \int_0^1 \nabla \cdot u_t(t) dt$$

- Then, we can learn such a vector field using a neural network $v_\theta(\mathbf{x}_t, t)$ by minimizing the following objective:

$$\mathcal{L} = \mathbb{E}_{t \sim \mathcal{U}[0,1]} \mathbb{E}_{\mathbf{x}_t \sim p_t} [\|v_\theta(\mathbf{x}_t, t) - u_t(\mathbf{x}_t)\|^2]$$

Flow Matching



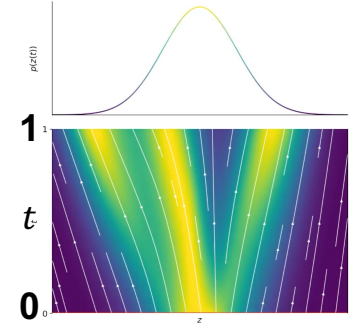
- Isn't this similar to what we've seen before?

$$\mathbb{E}_{t \sim \mathcal{U}[0,1]} \mathbb{E}_{\mathbf{x}_t \sim p_t} [\|v_\theta(\mathbf{x}_t, t) - u_t(\mathbf{x}_t)\|^2]$$

- **Noise prediction** network in diffusion models!

$$\mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), t > 1, q(\mathbf{x}_t | \mathbf{x}_0)} [\|\hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, t) - \boldsymbol{\epsilon}_t\|^2]$$

Flow Matching



But wait. Do we know $u_t(\mathbf{x}_t)$?

直接學習 marginal vector field $u_t(\mathbf{x}_t)$ 很困難，因為不知道真實的 marginal distribution $p_t(\mathbf{x}_t)$ ，而且需要解決複雜的 Fokker-Planck equation

No. Then, how can we minimize the objective?

Conditional Probability Paths

解決方法：改為學習 **conditional** vector field $u_t(\mathbf{x}_t|\mathbf{x}_1)$

Let's think about the **conditional probability distribution** $p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)$ such that:

$$p_t(\mathbf{x}_t) = \int p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)p_1(\mathbf{x}_1) d\mathbf{x}_1$$

當 $t = 0$ 時，conditional distribution 與 $\mathbf{x}_1$ 無關，不管終點在哪，確保了起點都從相同的 base distribution (如 standard normal) 開始 sample

Constraint 1: 起點 ($t = 0$)

$$p_{0|1}(\mathbf{x}_0|\mathbf{x}_1) = p_0(\mathbf{x}_0)$$

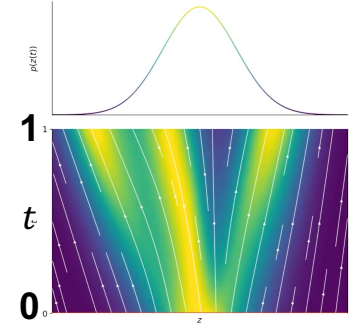
Constraint 2: 終點 ($t = 1$)

$$p_{1|1}(\mathbf{x}_1|\mathbf{x}_1) = \mathcal{N}(\mathbf{x}_1, \sigma_{\min}^2 \mathbf{I})$$

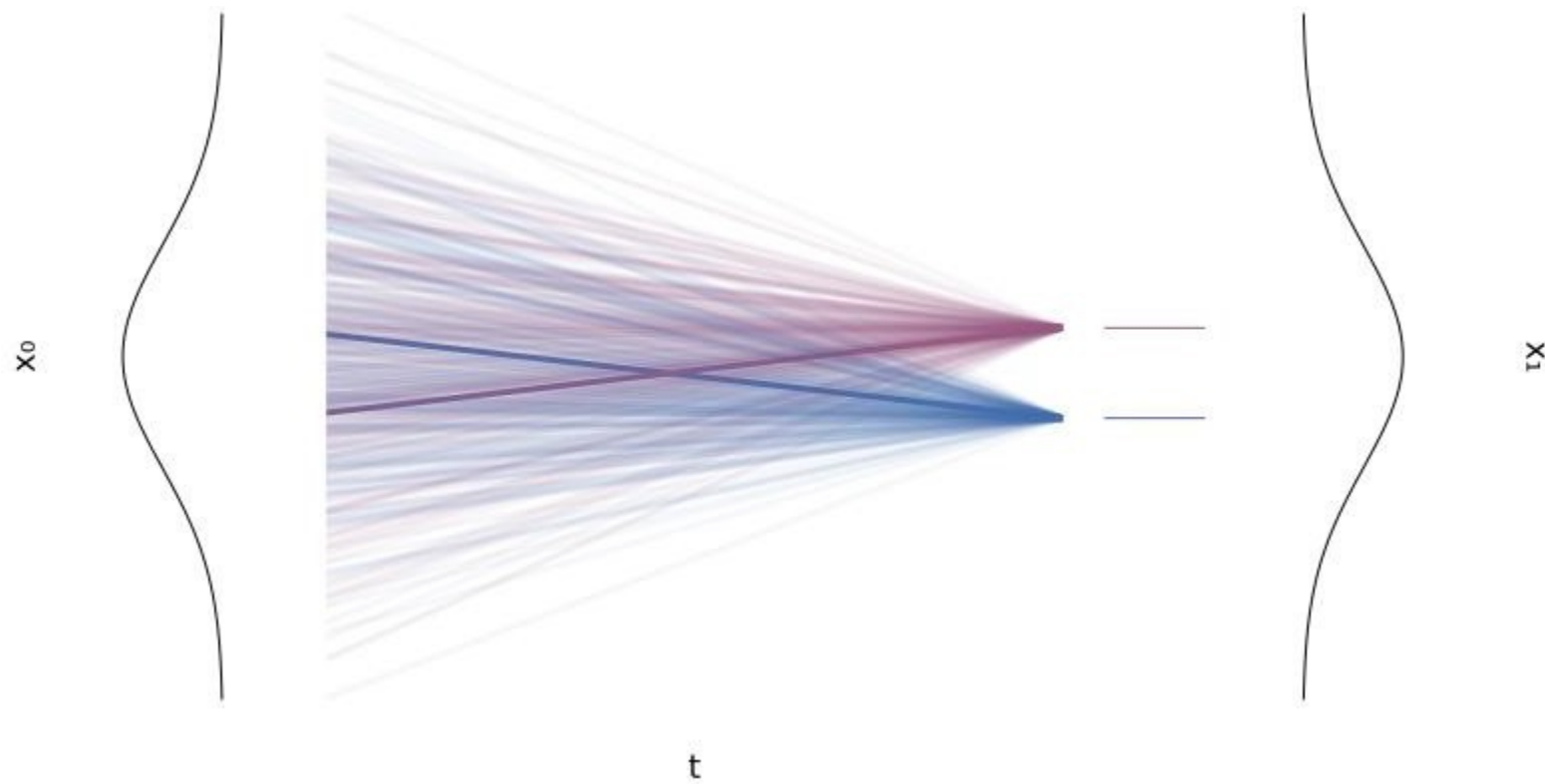
當 $t = 1$ 時，conditional distribution 高度集中在 $\mathbf{x}_1$ 附近， $\sigma_{\min}$ 很小，幾乎是 delta function，讓路徑幾乎確定可以到達目標點 $\mathbf{x}_1$

where $\sigma_{\min}$ is a very small value.

$\sigma_{\min} > 0$ 是一個必要的妥協：**理論上**：我們想要 delta 函數（完美到達），**實際上**：我們需要小但非零的 variance 這個小的 $\sigma_{\min}$ 就像是 regularization，足夠小以接近理想行為，但足夠大以保持數學和數值的良好性質



Conditional Probability Paths



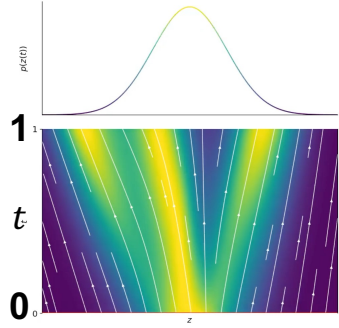
Conditional Probability Paths

- Isn't this similar to what we've seen before?

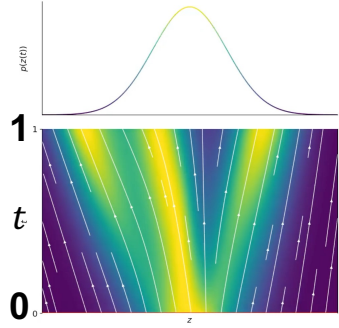
$$p_{t|1}(\mathbf{x}_t | \mathbf{x}_1)$$

- **Forward jump** in diffusion model!

$$q(\mathbf{x}_t | \mathbf{x}_0)$$



Conditional Probability Paths



- There can be many such **conditional probability paths**.
- Let's consider the following family of normal distributions:

$$p_{t|1}(\mathbf{x}_t | \mathbf{x}_1) = \mathcal{N}(\mu_t(\mathbf{x}_1), \sigma_t^2(\mathbf{x}_1)\mathbf{I})$$

such that

$$\mu_0(\mathbf{x}_1) = 0 \text{ and } \sigma_0(\mathbf{x}_1) = 1$$

$$\mu_1(\mathbf{x}_1) = \mathbf{x}_1 \text{ and } \sigma_1(\mathbf{x}_1) = \sigma_{\min}$$

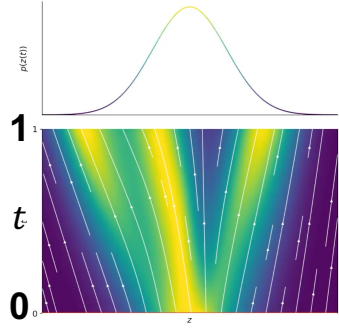
Conditional Vector Field

- From Fokker–Planck equation:

$$\frac{\partial p_t(\mathbf{x}_t)}{\partial t} = -\nabla \cdot [u_t(\mathbf{x}_t)p_t(\mathbf{x}_t)]$$

- Consider the **conditional vector field** $u_t(\mathbf{x}_t|\mathbf{x}_1)$:

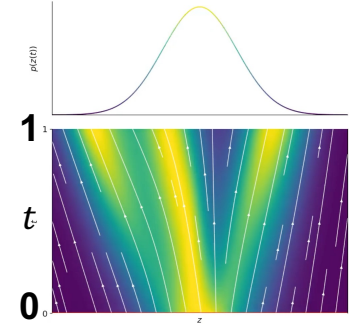
$$\frac{\partial p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)}{\partial t} = -\nabla \cdot [u_t(\mathbf{x}_t|\mathbf{x}_1)p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)]$$



Conditional Vector Fields

- Are the **conditional vector fields** unique given the conditional probability paths?
- No. There is also an **infinite number** of conditional vector fields that can generate the given conditional probability paths.

Conditional Vector Fields



Let's consider the following combination (for simplicity).

Conditional probability paths $p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)$: 定義了從 noise 到 data point $\mathbf{x}_1$ 的 probability paths

$$p_{t|1}(\mathbf{x}_t|\mathbf{x}_1) = \mathcal{N}(\mu_t(\mathbf{x}_1), \sigma_t^2(\mathbf{x}_1)\mathbf{I})$$

Conditional flow map $\phi_t(\mathbf{x}_t|\mathbf{x}_1)$: 這是 **deterministic** mapping，將 standard normal 變換到 conditional distribution 積分結果、global mapping、完整路徑

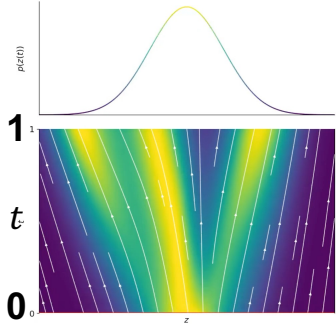
$$\phi_t(\mathbf{x}_t|\mathbf{x}_1) = \sigma_t(\mathbf{x}_1)\mathbf{x}_t + \mu_t(\mathbf{x}_1)$$

Conditional vector field $u_t(\mathbf{x}_t|\mathbf{x}_1)$: 這是 ODE 的 vector field
瞬時速度、local direction、在每個點的速度向量

對時間 t 偏微分

$$u_t(\mathbf{x}_t|\mathbf{x}_1) = \frac{\sigma'_t(\mathbf{x}_1)}{\sigma_t(\mathbf{x}_1)}(\mathbf{x}_t - \mu_t(\mathbf{x}_1)) + \mu'_t(\mathbf{x}_1)$$

Conditional Vector Fields



- The flow map $\phi_t(\mathbf{x}_t|\mathbf{x}_1)$ pushes the base distribution p_0 to $p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)$:

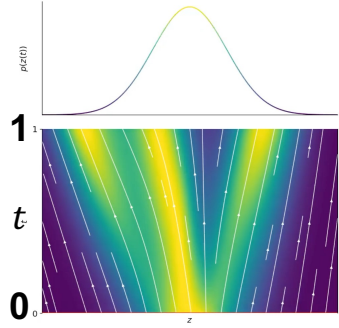
$$p_{t|1}(\mathbf{x}_t|\mathbf{x}_1) = [\phi_t]_* p_0(\mathbf{x}_t)$$

- Thus, given $\mathbf{x}_1$, sampling from $p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)$ is the same as:

$$\mathbf{x}_0 \sim p_0$$

$$\mathbf{x}_t = \phi_t(\mathbf{x}_0|\mathbf{x}_1)$$

Conditional Flow Matching



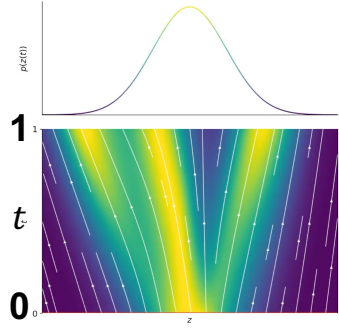
Why do we care about the *conditional vector field* $u_t(\mathbf{x}_t|\mathbf{x}_1)$?

Important Fact 1.

The marginal vector field $u_t(\mathbf{x}_t)$ can be expressed in term of the conditional vector field $u_t(\mathbf{x}_t|\mathbf{x}_1)$ as follows:

$$u_t(\mathbf{x}_t) = \int u_t(\mathbf{x}_t|\mathbf{x}_1) \frac{p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)p_1(\mathbf{x}_1)}{p_t(\mathbf{x}_t)} d\mathbf{x}_1$$

Conditional Flow Matching



Important Fact 2.

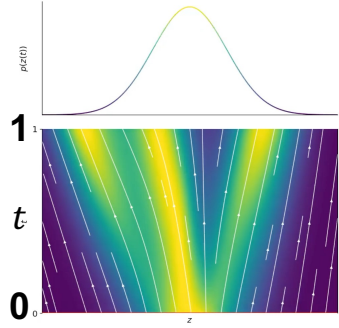
The loss

$$\mathcal{L} = \mathbb{E}_{t \sim \mathcal{U}[0,1]} \mathbb{E}_{\mathbf{x}_t \sim p_t} [\|v_\theta(\mathbf{x}_t, t) - u_t(\mathbf{x}_t)\|^2]$$

can be replaced with an **equivalent** loss regressing the **conditional vector field**:

$$\mathcal{L} = \mathbb{E}_{t \sim \mathcal{U}[0,1]} \mathbb{E}_{\mathbf{x}_1 \sim p_1, \mathbf{x}_t \sim p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)} [\|v_\theta(\mathbf{x}_t, t) - u_t(\mathbf{x}_t|\mathbf{x}_1)\|^2]$$

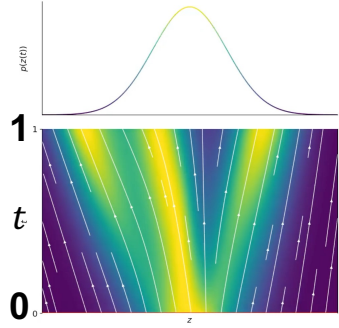
Conditional Flow Matching



$$\mathcal{L} = \mathbb{E}_{t \sim \mathcal{U}[0,1]} \mathbb{E}_{\mathbf{x}_1 \sim p_1, \mathbf{x}_t \sim p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)} [\|v_\theta(\mathbf{x}_t, t) - u_t(\mathbf{x}_t|\mathbf{x}_1)\|^2]$$

While we don't know $u_t(\mathbf{x}_t)$, we can compute $u_t(\mathbf{x}_t|\mathbf{x}_1)$ depending on our choice of $p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)$.

Conditional Flow Matching



$$\mathcal{L} = \mathbb{E}_{t \sim \mathcal{U}[0,1]} \mathbb{E}_{\mathbf{x}_1 \sim p_1, \mathbf{x}_t \sim p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)} [\|v_\theta(\mathbf{x}_t, t) - u_t(\mathbf{x}_t|\mathbf{x}_1)\|^2]$$

But now how do we sample from $p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)$?

$$\phi_t(x_t|x_1) = \sigma_t(x_1)x_t + \mu_t(x_1).$$

$$1. p_{t|1}(x_t|x_1) = [\phi_t]_* p_0(x_t)$$

• Thus,
t

$$x_0 \sim p_0$$

0

$$x_t = \phi_t(x_0)$$

Conditional Flow Matching

Recall the conditional flow map $\phi_t(x_t|x_1)$:

$$\phi_t(x_t|x_1) = \sigma_t(x_1)x_t + \mu_t(x_1).$$

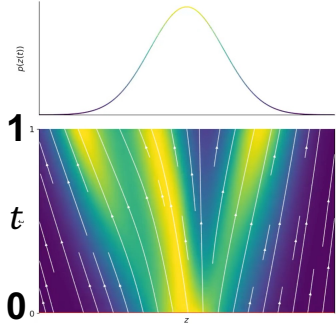
- $p_{t|1}(x_t|x_1) = [\phi_t]_* p_0(x_t)$

- Thus,

$$x_0 \sim p_0$$

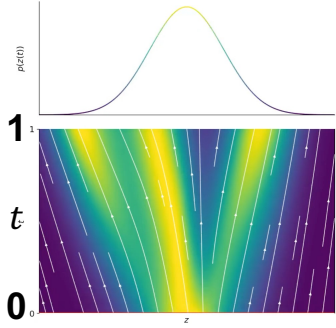
$$x_t = \phi_t(x_0)$$

Conditional Flow Matching



$$\mathcal{L} = \mathbb{E}_{t \sim \mathcal{U}[0,1]} \mathbb{E}_{\mathbf{x}_1 \sim p_1, \mathbf{x}_0 \sim p_0} [\|v_\theta(\phi_t(\mathbf{x}_0), t) - u_t(\phi_t(\mathbf{x}_0) | \mathbf{x}_1)\|^2]$$

Conditional Probability Paths



- Recall the following family of normal distributions:

$$p_{t|1}(\mathbf{x}_t|\mathbf{x}_1) = \mathcal{N}(\mu_t(\mathbf{x}_1), \sigma_t^2(\mathbf{x}_1)\mathbf{I})$$

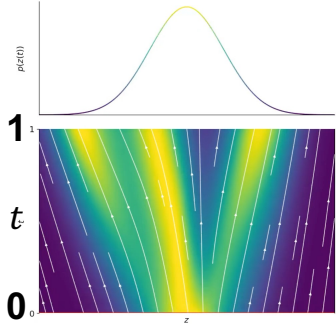
such that

$$\mu_0(\mathbf{x}_1) = 0 \text{ and } \sigma_0(\mathbf{x}_1) = 1$$

$$\mu_1(\mathbf{x}_1) = \mathbf{x}_1 \text{ and } \sigma_1(\mathbf{x}_1) = \sigma_{\min}$$

- What should be $\mu_t(\mathbf{x}_1)$ and $\sigma_t(\mathbf{x}_1)$?

Recall: VP-SDE vs. VE-SDE



- **DDPM** is a special case (**Variance Preserving**) when the forward transitional distribution $p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)$ is defined as

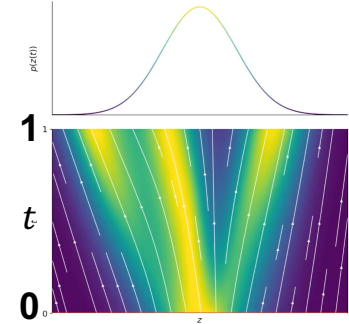
$$p_{t|1}(\mathbf{x}_t|\mathbf{x}_1) = \mathcal{N}(\alpha_{1-t}\mathbf{x}_1, (1 - \alpha_{1-t}^2)\mathbf{I})$$

- **SMLD** (Score Matching with Langevin Dynamics) is another case (**Variance Exploding**) when the forward transitional distribution $p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)$ is:

$$p_{t|1}(\mathbf{x}_t|\mathbf{x}_1) = \mathcal{N}(\mathbf{x}_1, \sigma_{1-t}^2\mathbf{I})$$

Be careful with the notation changes.

Quiz 1



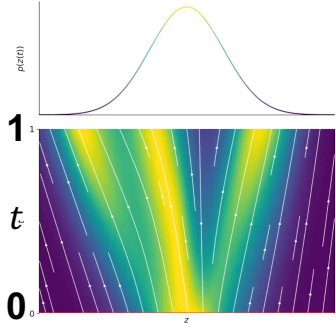
In the **Variance Preserving (VP)** case, the forward jump was

$$p_{t|1}(\mathbf{x}_t|\mathbf{x}_1) = \mathcal{N}(\alpha_{1-t}\mathbf{x}_1, (1 - \alpha_{1-t}^2)\mathbf{I}) \quad \text{也就是 DDPM}$$

Q. What is $u_t(\mathbf{x}_t|\mathbf{x}_1)$?

Hint. $u_t(\mathbf{x}_t|\mathbf{x}_1) = \frac{\sigma'_t(\mathbf{x}_1)}{\sigma_t(\mathbf{x}_1)} (\mathbf{x}_t - \mu_t(\mathbf{x}_1)) + \mu'_t(\mathbf{x}_1)$

Answer 1



- $\mu_t(\mathbf{x}_1) = \alpha_{1-t}\mathbf{x}_1 \Rightarrow \frac{d}{dt}\mu_t(\mathbf{x}_1) = -\alpha'_{1-t}\mathbf{x}_1$

- $\sigma_t(\mathbf{x}_1) = \sqrt{1 - \alpha_{1-t}^2} \Rightarrow \frac{d}{dt}\sigma_t(\mathbf{x}_1) = \frac{\alpha_{1-t}\alpha'_{1-t}}{\sqrt{1 - \alpha_{1-t}^2}}$

$$u_t(\mathbf{x}_t|\mathbf{x}_1) = \frac{\sigma'_t(\mathbf{x}_1)}{\sigma_t(\mathbf{x}_1)}(\mathbf{x}_t - \mu_t(\mathbf{x}_1)) + \mu'_t(\mathbf{x}_1)$$

$$= \frac{\alpha_{1-t}\alpha'_{1-t}}{\sqrt{1 - \alpha_{1-t}^2}} \frac{1}{\sqrt{1 - \alpha_{1-t}^2}}(\mathbf{x}_t - \alpha_{1-t}\mathbf{x}_1) - \alpha'_{1-t}\mathbf{x}_1$$

$$= \frac{\alpha_{1-t}\alpha'_{1-t}\mathbf{x}_t - \alpha_{1-t}^2\alpha'_{1-t}\mathbf{x}_1 - (1 - \alpha_{1-t}^2)\alpha'_{1-t}\mathbf{x}_1}{1 - \alpha_{1-t}^2} = \frac{\alpha'_{1-t}(\alpha_{1-t}\mathbf{x}_t - \mathbf{x}_1)}{1 - \alpha_{1-t}^2}$$

Diffusion vs. Flow Matching

Diffusion 和 Flow Matching 不是完全不同的方法，而是**同一過程的不同視角**

- Diffusion : SDE 視角 (stochastic)
- Flow Matching : ODE 視角 (deterministic)

既然任何 forward process 都可以轉換成 flow，每種 diffusion 變體都有對應的 flow matching 版本

Flow Matching 提供了另一種**更直接的方式來學習 noise -> data 路徑**

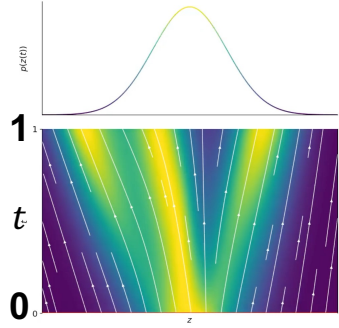
Quiz 2

Let's consider a simpler case:

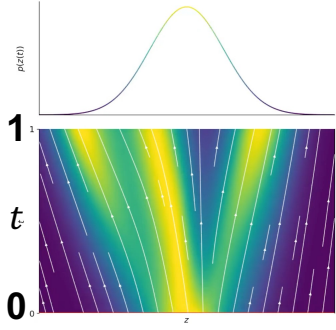
$$p_{t|1}(\mathbf{x}_t | \mathbf{x}_1) = \mathcal{N}(t\mathbf{x}_1, (1 - (1 - \sigma_{\min})t)\mathbf{I})$$

Q. What is $u_t(\mathbf{x}_t | \mathbf{x}_1)$?

Hint. $u_t(\mathbf{x}_t | \mathbf{x}_1) = \frac{\sigma'_t(\mathbf{x}_1)}{\sigma_t(\mathbf{x}_1)} (\mathbf{x}_t - \mu_t(\mathbf{x}_1)) + \mu'_t(\mathbf{x}_1)$



Answer 2

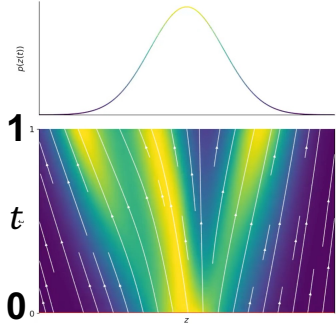


- $\mu_t(\mathbf{x}_1) = t\mathbf{x}_1 \Rightarrow \frac{d}{dt}\mu_t(\mathbf{x}_1) = \mathbf{x}_1$

- $\sigma_t(\mathbf{x}_1) = 1 - (1 - \sigma_{\min})t \Rightarrow \frac{d}{dt}\sigma_t(\mathbf{x}_1) = -(1 - \sigma_{\min})$

$$\begin{aligned} u_t(\mathbf{x}_t|\mathbf{x}_1) &= \frac{\sigma'_t(\mathbf{x}_1)}{\sigma_t(\mathbf{x}_1)} (\mathbf{x}_t - \mu_t(\mathbf{x}_1)) + \mu'_t(\mathbf{x}_1) \\ &= \frac{-(1 - \sigma_{\min})\mathbf{x}_t + (1 - \sigma_{\min})t\mathbf{x}_1}{1 - (1 - \sigma_{\min})t} + \frac{\mathbf{x}_1 - (1 - \sigma_{\min})t\mathbf{x}_1}{1 - (1 - \sigma_{\min})t} \\ &= \frac{\mathbf{x}_1 - (1 - \sigma_{\min})\mathbf{x}_t}{1 - (1 - \sigma_{\min})t} \end{aligned}$$

Conditional Flow Matching



Conditional probability paths $p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)$:

$$p_{t|1}(\mathbf{x}_t|\mathbf{x}_1) = \mathcal{N}(t\mathbf{x}_1, (1 - (1 - \sigma_{\min})t)\mathbf{I})$$

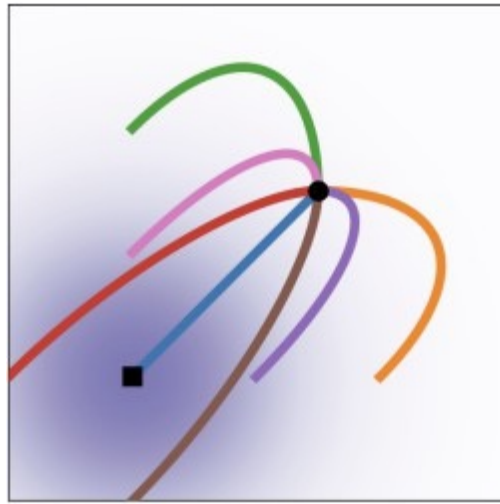
Conditional flow map $\phi_t(\mathbf{x}_t|\mathbf{x}_1)$:

$$\phi_t(\mathbf{x}_t|\mathbf{x}_1) = (1 - (1 - \sigma_{\min})t)\mathbf{x}_t + t\mathbf{x}_1$$

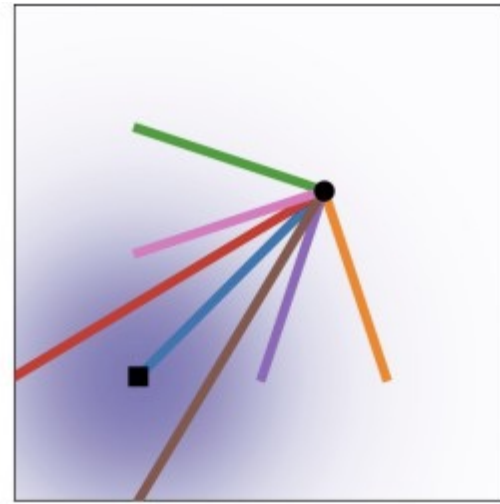
Conditional vector field $u_t(\mathbf{x}_t|\mathbf{x}_1)$:

$$u_t(\mathbf{x}_t|\mathbf{x}_1) = \frac{\mathbf{x}_1 - (1 - \sigma_{\min})\mathbf{x}_t}{1 - (1 - \sigma_{\min})t}$$

Conditional Flow Matching

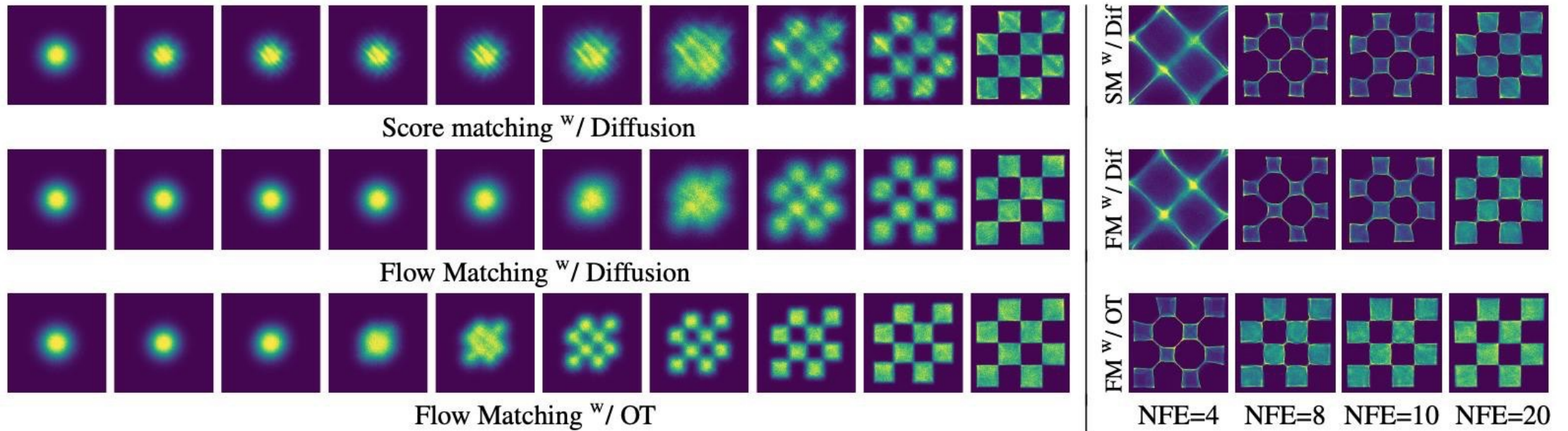


Diffusion

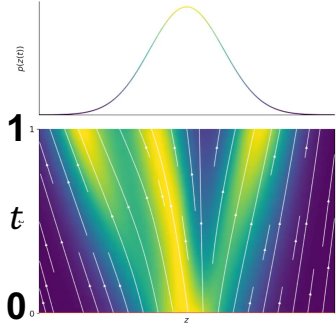


OT

Conditional Flow Matching



Push-Forward Operator



The distribution $p_1(\mathbf{x}_1)$ induced by the following sampling procedure:

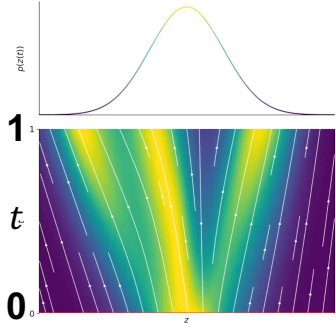
$$\mathbf{x}_0 \sim p_0$$

$$\mathbf{x}_1 = \phi(\mathbf{x}_0)$$

Is expressed using the push-forward operator $[\phi]_*$ as:

$$p_1(\mathbf{x}_1) = [\phi]_* p_0(\mathbf{x}_1) = p_0(\phi^{-1}(\mathbf{x}_1)) \left| \det \left(\frac{\partial \phi^{-1}(\mathbf{x}_1)}{\partial \mathbf{x}_1} \right) \right|$$

Velocity Field and Probability Paths



Given a **flow map** $\phi_t(\mathbf{x}_t)$ mapping p_0 to p_1 ,

- A **velocity field** $u_t(t)$ is a derivative of the flow map:

$$u_t(t) = \frac{d}{dt} \mathbf{x}_t$$

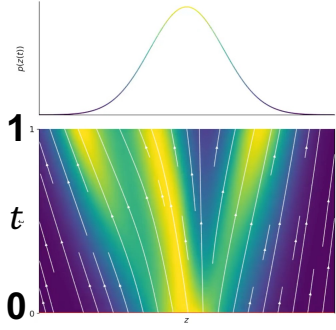
- **Probability paths** $p_t(\mathbf{x}_t)$ can also be derived from the velocity field based on the Fokker–Planck equation:

$$\frac{\partial p_t(\mathbf{x}_t)}{\partial t} = \nabla \cdot [u_t(\mathbf{x}_t)p_t(\mathbf{x}_t)]$$

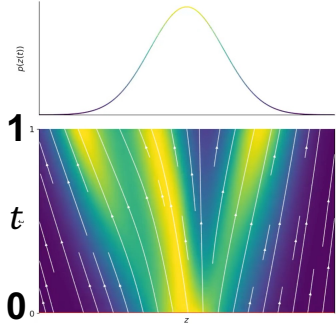
Flow Matching

Given a **vector field** $u_t(\mathbf{x}_t)$, learn the vector field using a neural network $v_\theta(\mathbf{x}_t, t)$ by minimizing the following objective:

$$\mathcal{L} = \mathbb{E}_{t \sim \mathcal{U}[0,1]} \mathbb{E}_{\mathbf{x}_t \sim p_t} [\|v_\theta(\mathbf{x}_t, t) - u_t(\mathbf{x}_t)\|^2]$$



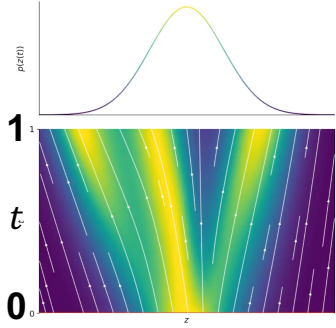
Conditional Vector Field



- Instead of a vector field $u_t(t)$, let's consider a **conditional vector field** $u_t(\mathbf{x}_t|\mathbf{x}_1)$.
- The following loss function based on the conditional vector field $u_t(\mathbf{x}_t|\mathbf{x}_1)$ is equivalent:

$$\mathcal{L} = \mathbb{E}_{t \sim \mathcal{U}[0,1]} \mathbb{E}_{\mathbf{x}_1 \sim p_1, \mathbf{x}_t \sim p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)} [\|v_\theta(\mathbf{x}_t, t) - u_t(\mathbf{x}_t|\mathbf{x}_1)\|^2]$$

Conditional Vector Fields



Which conditional vector field could we consider?

Conditional probability paths $p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)$:

$$p_{t|1}(\mathbf{x}_t|\mathbf{x}_1) = \mathcal{N}(\mu_t(\mathbf{x}_1), \sigma_t^2(\mathbf{x}_1)\mathbf{I})$$

Conditional flow map $\phi_t(\mathbf{x}_t|\mathbf{x}_1)$:

$$\phi_t(\mathbf{x}_t|\mathbf{x}_1) = \sigma_t(\mathbf{x}_1)\mathbf{x}_t + \mu_t(\mathbf{x}_1)$$

Conditional vector field $u_t(\mathbf{x}_t|\mathbf{x}_1)$:

$$u_t(\mathbf{x}_t|\mathbf{x}_1) = \frac{\sigma_t'(\mathbf{x}_1)}{\sigma_t(\mathbf{x}_1)} (\mathbf{x}_t - \mu_t(\mathbf{x}_1)) + \mu_t'(\mathbf{x}_1)$$

Quiz

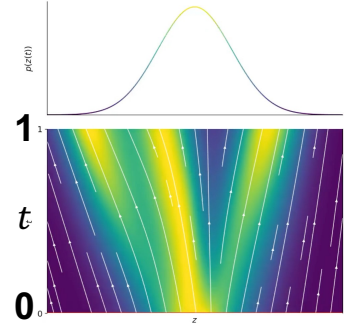
Let's consider the following mean and variance functions:

$$\mu_t(\mathbf{x}_1) = t\mathbf{x}_1$$

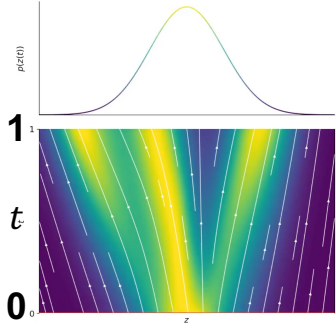
$$\sigma_t(\mathbf{x}_1) = 1 - (1 - \sigma_{\min})t$$

Q. What are the following in this case?

- **Conditional probability paths** $p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)$
- **Conditional flow map** $\phi_t(\mathbf{x}_t|\mathbf{x}_1)$
- **Conditional vector field** $u_t(\mathbf{x}_t|\mathbf{x}_1)$



Conditional Flow Matching



Conditional probability paths $p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)$:

$$p_{t|1}(\mathbf{x}_t|\mathbf{x}_1) = \mathcal{N}(t\mathbf{x}_1, (1 - (1 - \sigma_{\min})t)\mathbf{I})$$

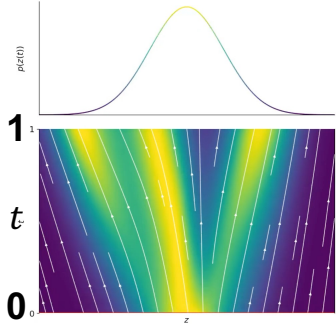
Flow map $\phi_t(\mathbf{x}_t)$ (Conditioned on $\mathbf{x}_1$):

$$\phi_t(\mathbf{x}_t|\mathbf{x}_1) = (1 - (1 - \sigma_{\min})t)\mathbf{x}_t + t\mathbf{x}_1$$

Conditional vector field $u_t(\mathbf{x}_t|\mathbf{x}_1)$:

$$u_t(\mathbf{x}_t|\mathbf{x}_1) = \frac{\mathbf{x}_1 - (1 - \sigma_{\min})\mathbf{x}_t}{1 - (1 - \sigma_{\min})t}$$

Conditional Flow Matching



Conditional probability paths $p_{t|1}(\mathbf{x}_t|\mathbf{x}_1)$:

$$p_{t|1}(\mathbf{x}_t|\mathbf{x}_1) = \mathcal{N}(t\mathbf{x}_1, (1-t)\mathbf{I})$$

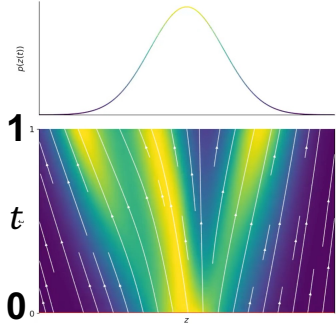
Conditional flow map $\phi_t(\mathbf{x}_t)$:

$$\phi_t(\mathbf{x}_t|\mathbf{x}_1) = (1-t)\mathbf{x}_t + t\mathbf{x}_1 \quad \text{Linear Interpolation!}$$

Conditional vector field $u_t(\mathbf{x}_t|\mathbf{x}_1)$:

$$u_t(\mathbf{x}_t|\mathbf{x}_1) = \frac{\mathbf{x}_1 - \mathbf{x}_t}{1-t}$$

Conditional Flow Matching

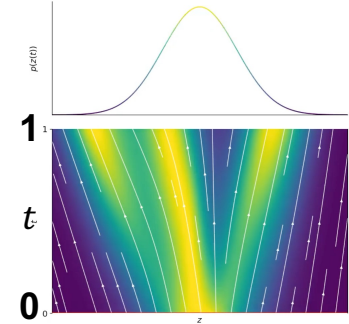


$$\mathcal{L} = \mathbb{E}_{t \sim \mathcal{U}[0,1]} \mathbb{E}_{\mathbf{x}_1 \sim p_1, \mathbf{x}_0 \sim p_0} [\|v_\theta(\phi_t(\mathbf{x}_0|\mathbf{x}_1), t) - u_t(\phi_t(\mathbf{x}_0|\mathbf{x}_1)|\mathbf{x}_1)\|^2]$$

1. Sample $\mathbf{x}_0 \sim p_0$ and $\mathbf{x}_1 \sim p_1$.
2. Compute $\mathbf{x}_t = \phi_t(\mathbf{x}_0|\mathbf{x}_1) = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1$ as a linear interpolation between $\mathbf{x}_0$ and $\mathbf{x}_1$.
3. Compute $u_t(\mathbf{x}_t|\mathbf{x}_1) = \frac{\mathbf{x}_1 - \mathbf{x}_t}{1 - t} = \mathbf{x}_1 - \mathbf{x}_0$.
4. Backpropagate through the regression loss.

Conditional Flow Matching vs. Diffusion

$$\mathcal{L} = \mathbb{E}_{t \sim \mathcal{U}[0,1]} \mathbb{E}_{\mathbf{x}_1 \sim p_1, \mathbf{x}_0 \sim p_0} [\|v_\theta(\phi_t(\mathbf{x}_0), t) - u_t(\phi_t(\mathbf{x}_0)|\mathbf{x}_1)\|^2]$$



Conditional Flow Matching

- Sample 兩個點
 - $\mathbf{x}_1 \sim p_{\text{data}}$ (data point/終點)
 - $\mathbf{x}_0 \sim p_0$ (noise/起點)
- Construct path : $\phi_t(\mathbf{x}_0) = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1$
- Predict vector field : $v_\theta(\mathbf{x}_t, t)$ 去逼近 $u_t(\mathbf{x}_t|\mathbf{x}_1) = \mathbf{x}_1 - \mathbf{x}_0$

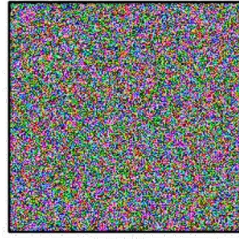
學習從 $\mathbf{x}_0$ 流向 $\mathbf{x}_1$ 的 vector field

Diffusion

- 只 sample data point $\mathbf{x}_1 \sim p_{\text{data}}$
- 正向加噪 : $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_1 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\varepsilon}$
- Predict noise : $\hat{\boldsymbol{\varepsilon}}_\theta(\mathbf{x}_t, t)$

學習如何去除添加的 noise

Flow matching

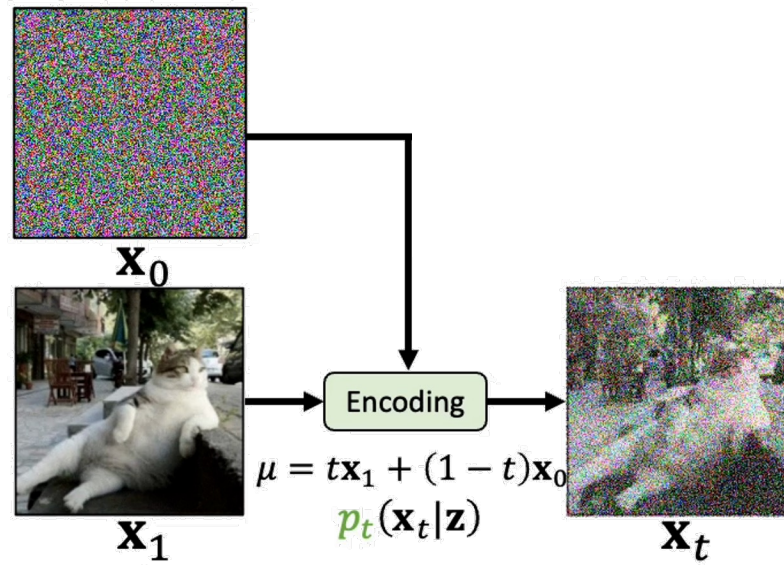


x_0

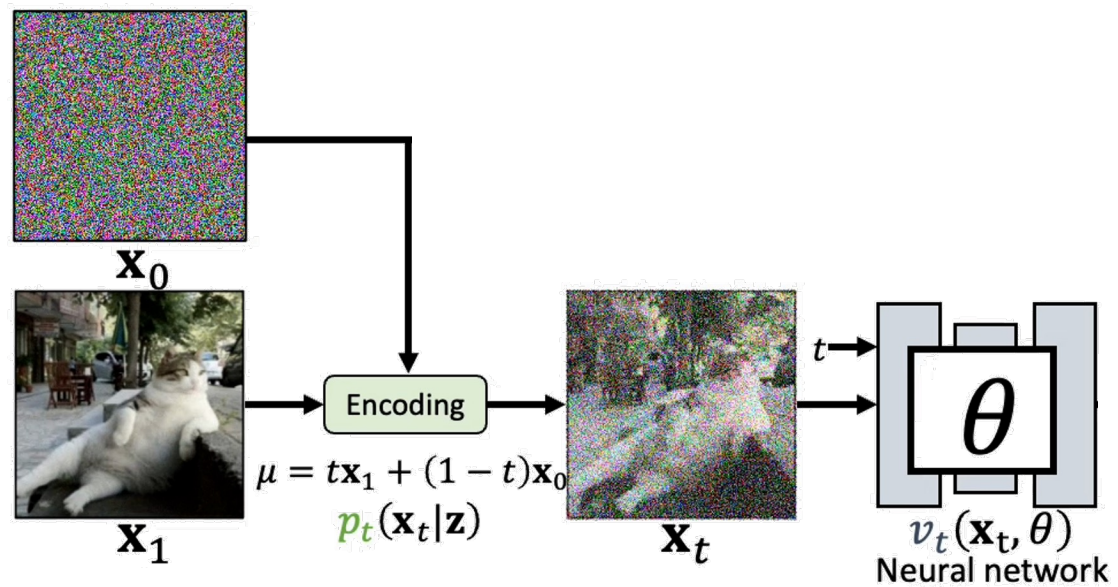


x_1

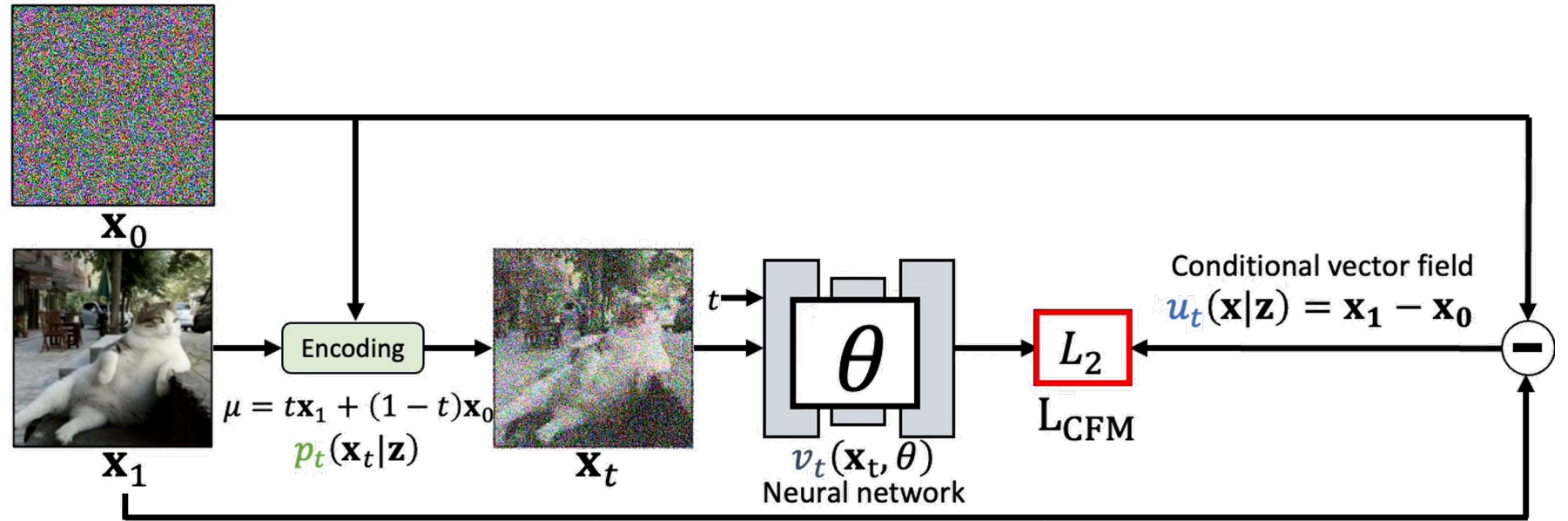
Flow matching



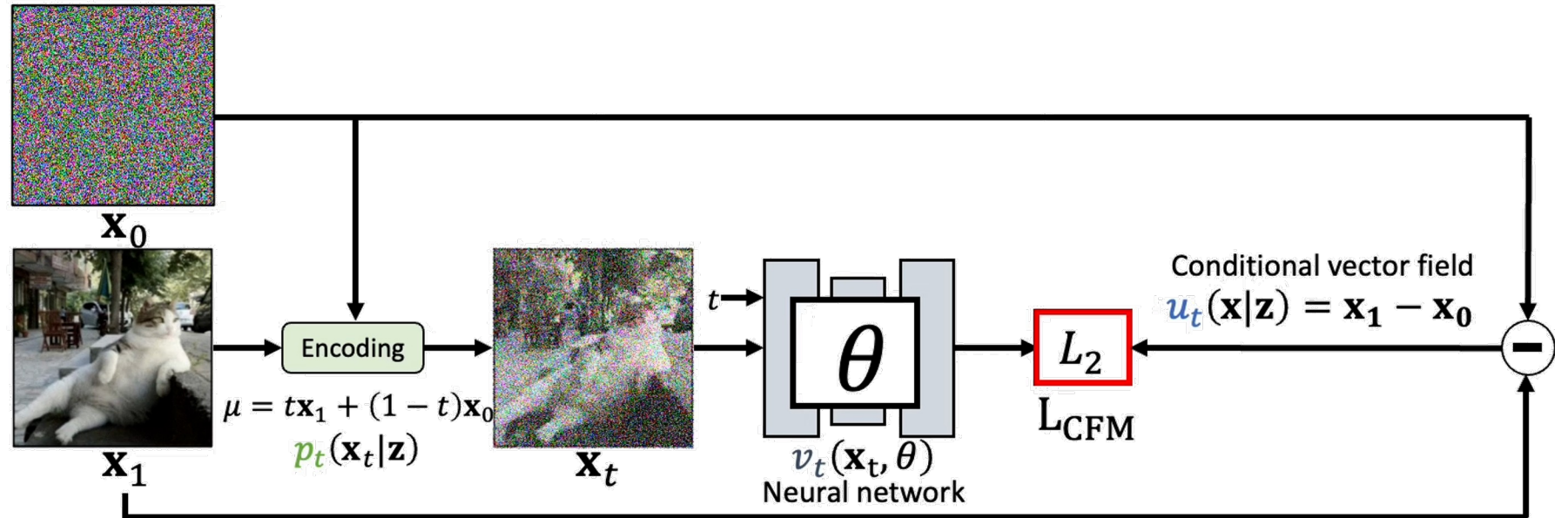
Flow matching



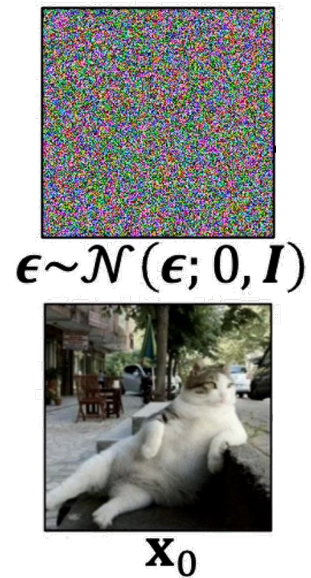
Flow matching



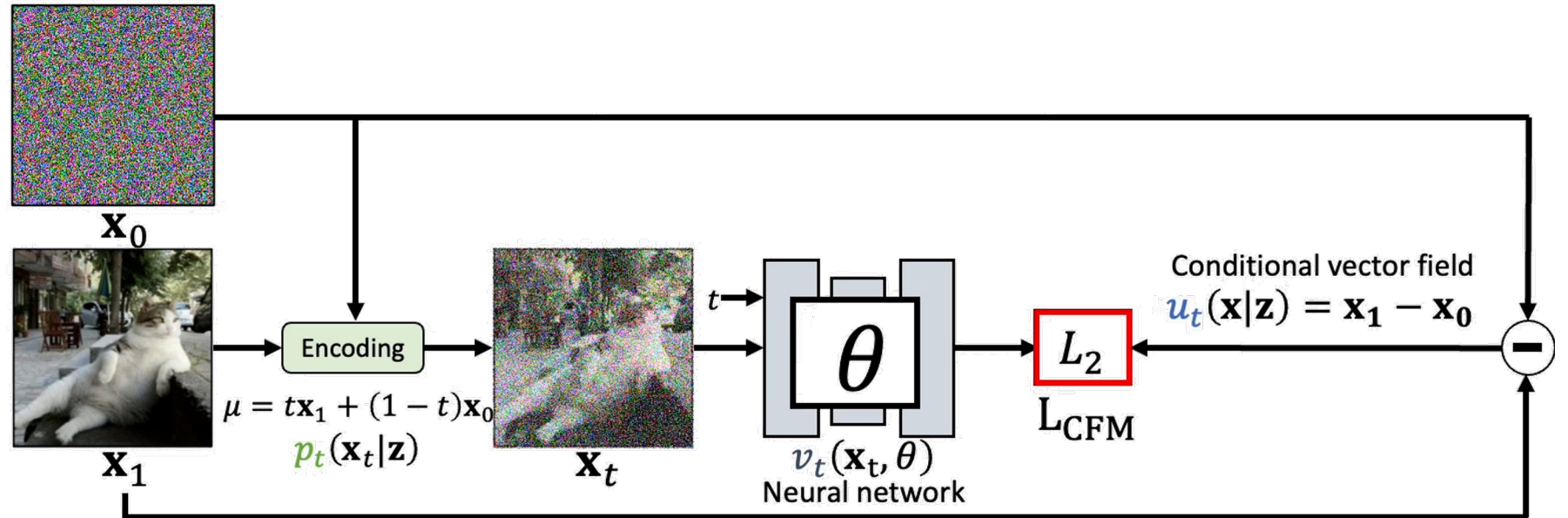
Flow matching



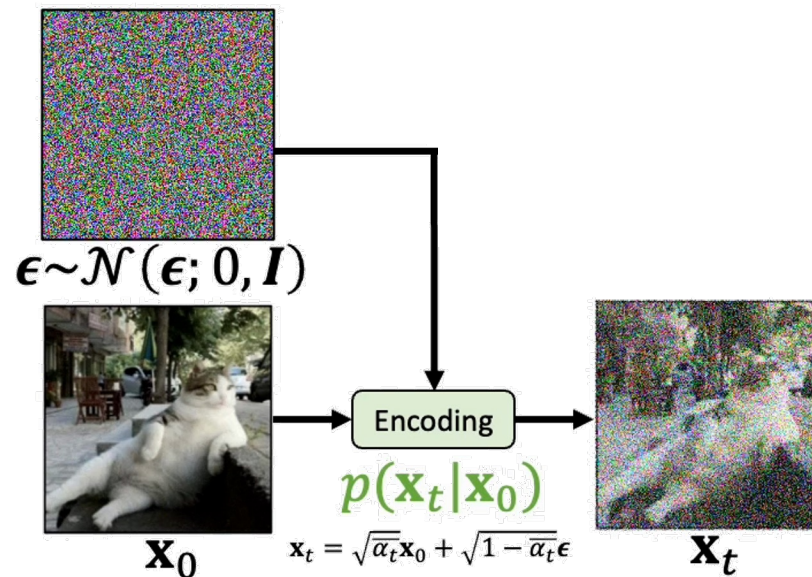
Diffusion



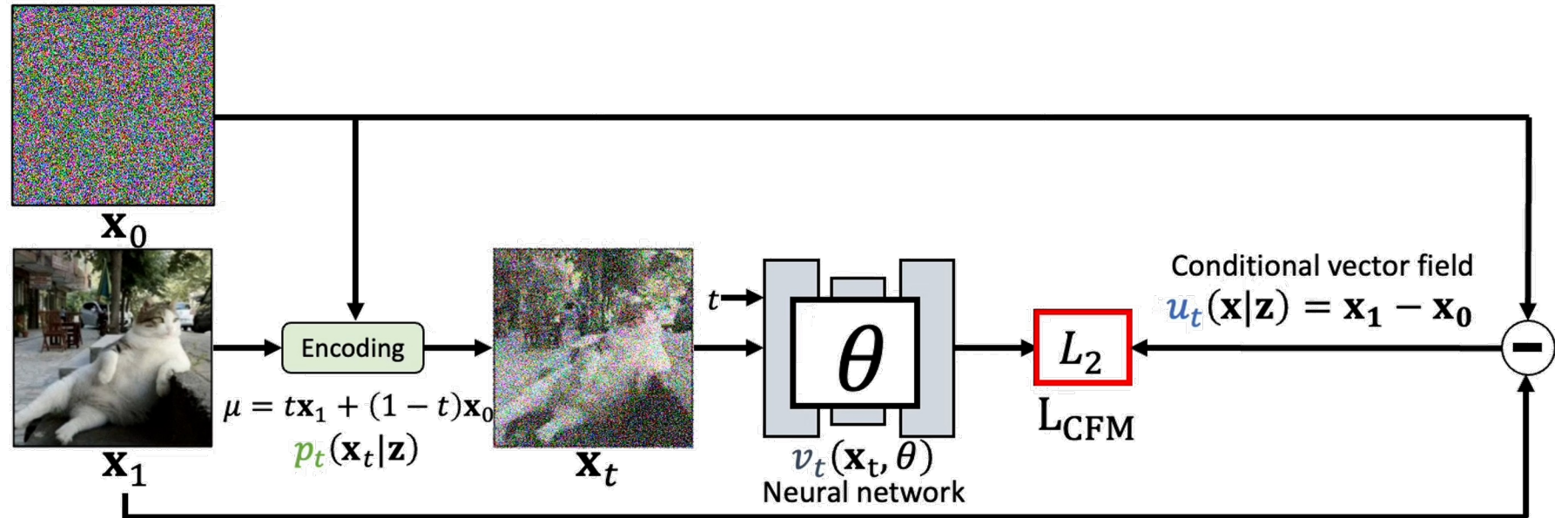
Flow matching



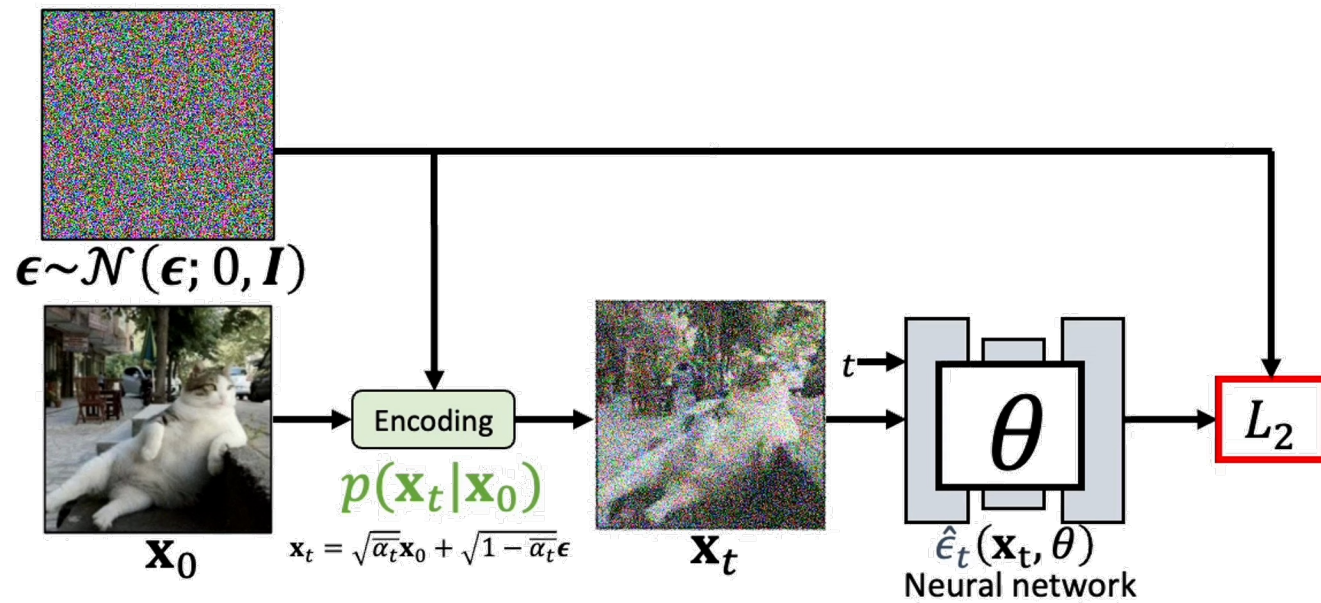
Diffusion



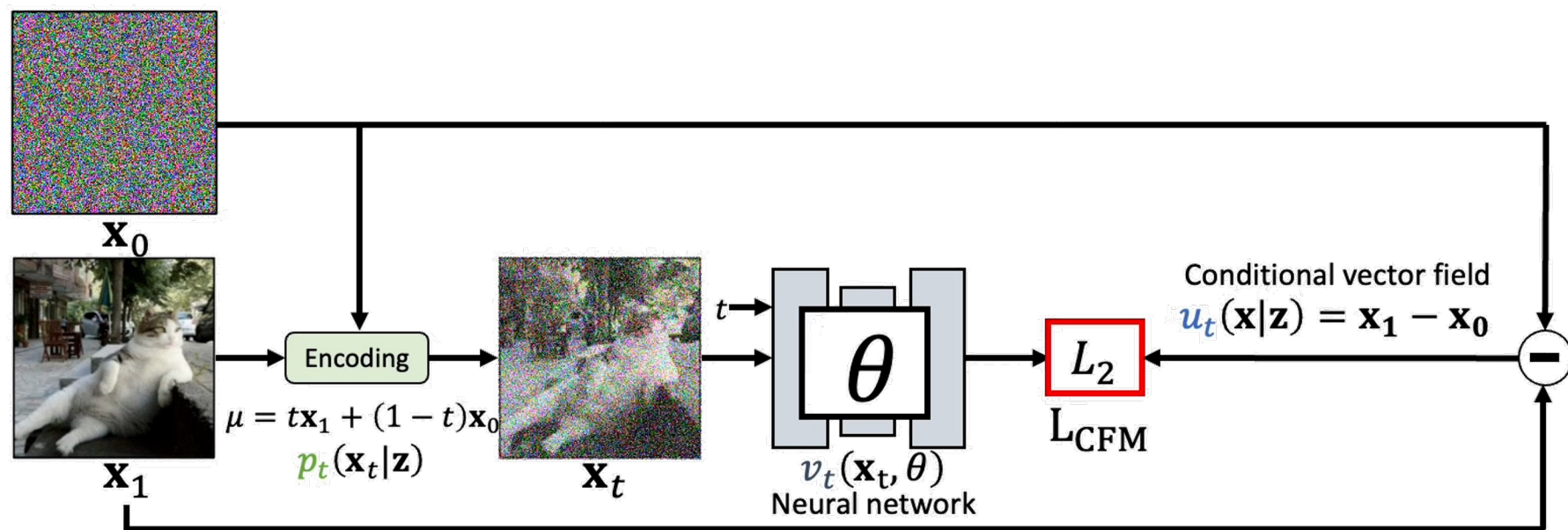
Flow matching



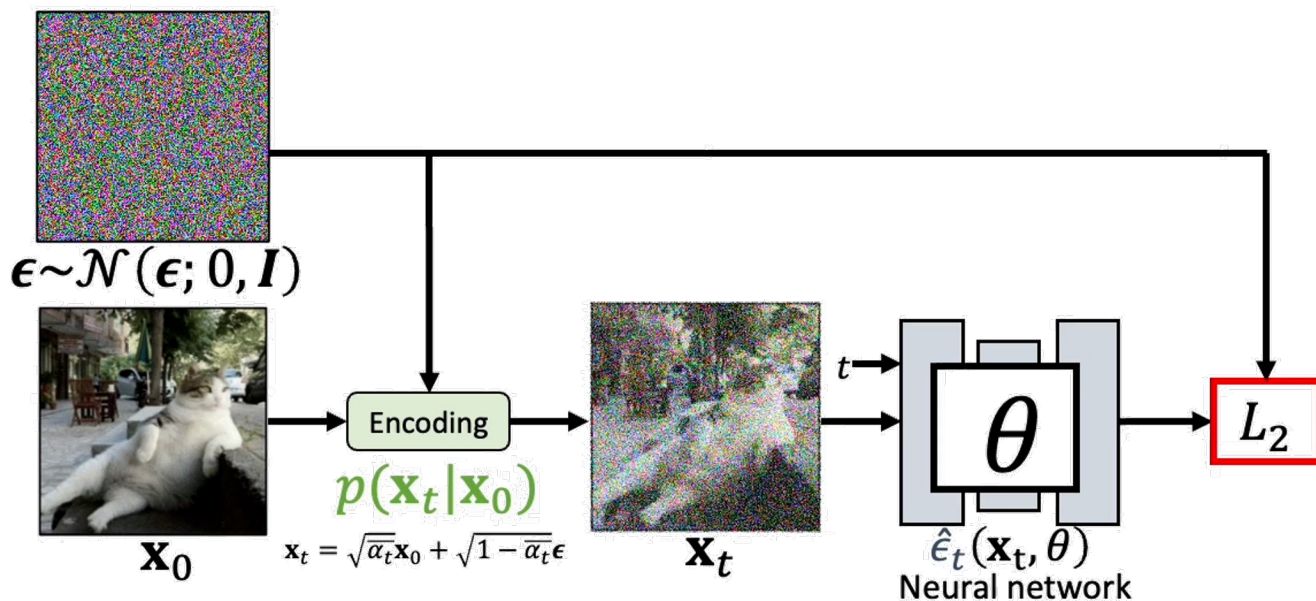
Diffusion



Flow matching



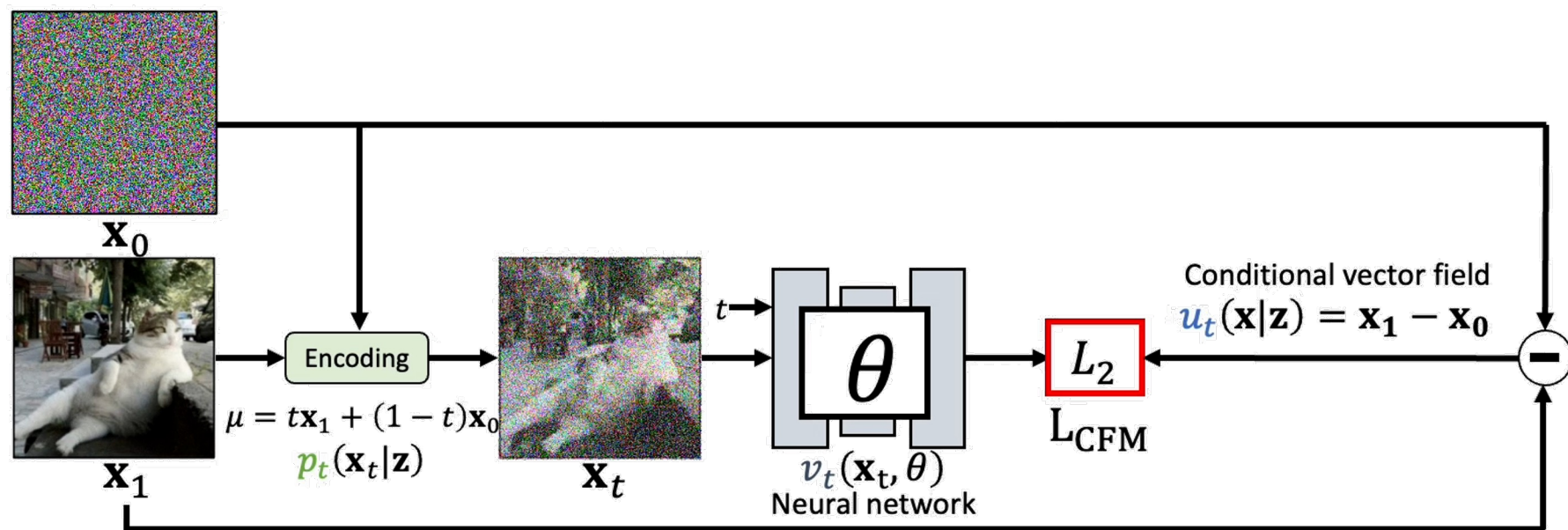
Diffusion



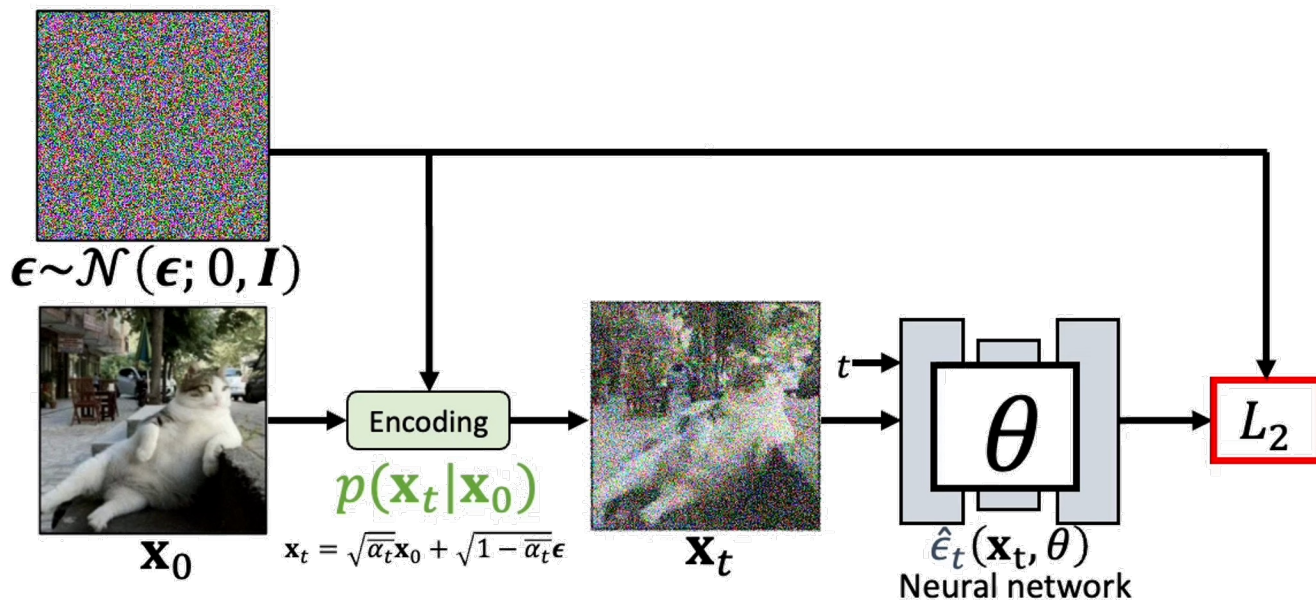
Diffusion 沒有辦法在有限的 time step 內透過正向加噪把影像變成 pure Gaussian noise

Flow matching 直接 regress base to target distribution 之間的移動/flow

Flow matching



Diffusion



Diffusion 沒有辦法在有限的 time step 內透過正向加噪把影像變成 pure Gaussian noise

Rectified Flow / Reflow

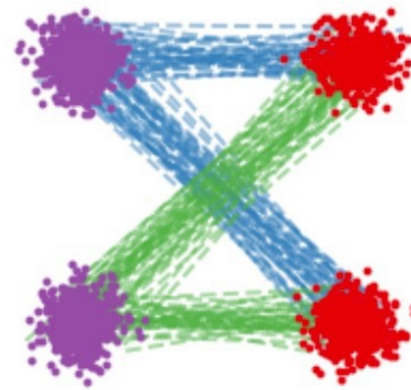
Liu et al., Rectified Flow: A Marginal Preserving Approach to Optimal Transport, 2022.

Liu et al., Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow, ICLR 2023.

Rectified Flow

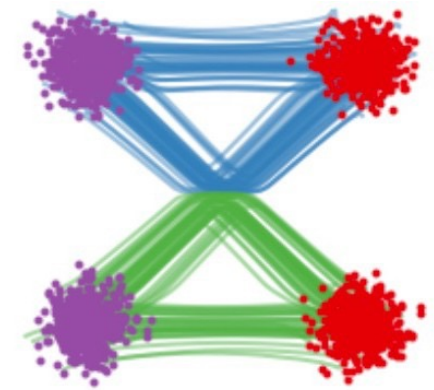
So, does this work well?

- Yes, but the outputs are not very good.
- The interesting thing is that it learns a vector field that generates trajectories that tend to **avoid crossing** each other (but not perfectly).



(a) Linear interpolation

$$X_t = tX_1 + (1 - t)X_0$$



(b) Rectified flow Z_t

induced by (X_0, X_1)

Rectified flow 軌跡不會交叉，主要是因為 ODE 的唯一性定理 (Uniqueness Theorem)

Rectified flow 定義了一個確定性的 ODE： $\frac{dx_t}{dt} = v(x_t, t)$

當 velocity field $v(x_t, t)$ 是 Lipschitz continuous 時，從不同起點出發的軌跡在相同時間點不會相交

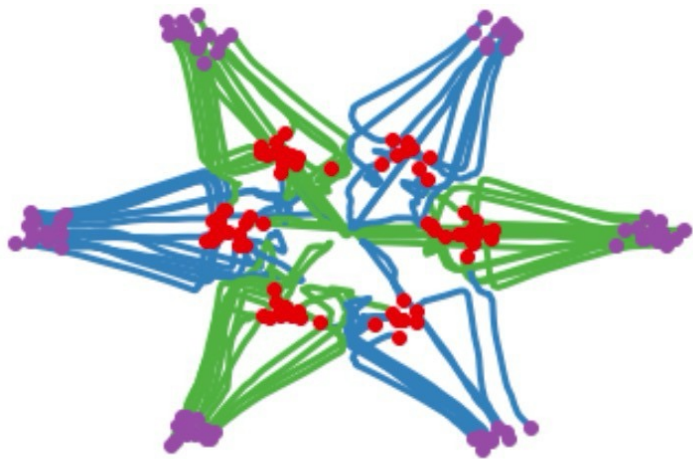
Reflow

A more interesting thing:

- Train the vector field network and call this the **first rectified flow** Z_1 .
- Then, train **another vector field network** (or **finetune** the previous one) while now sampling $\mathbf{x}_0$ and $\mathbf{x}_1$ using the first rectified flow Z_1 :

$$\mathcal{L} = \mathbb{E}_{t \sim \mathcal{U}[0,1]} \mathbb{E}_{(\mathbf{x}_0, \mathbf{x}_1) \sim Z_1} [\|\mathbf{v}_\theta(\mathbf{x}_t, t) - (\mathbf{x}_1 - \mathbf{x}_0)\|^2]$$

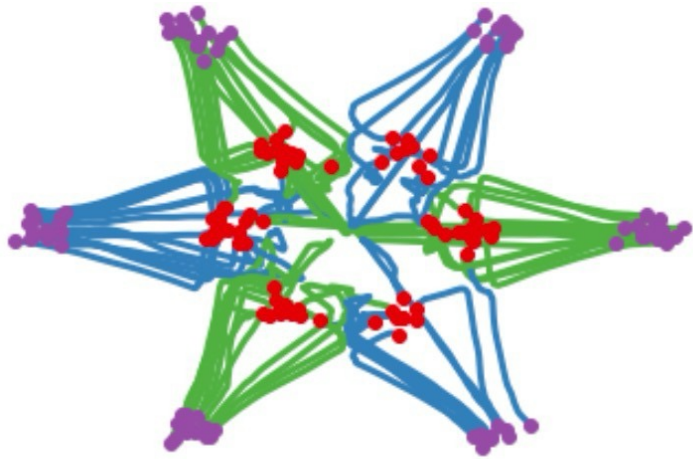
Reflow



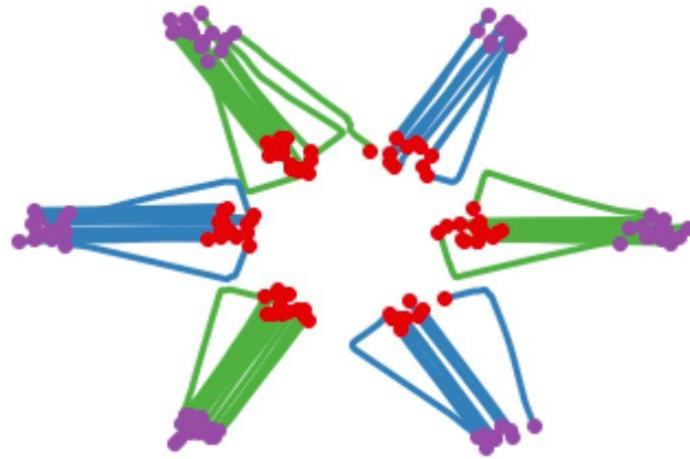
(a) The 1st rectified flow Z^1

Reflow

Now we get straighter trajectories!



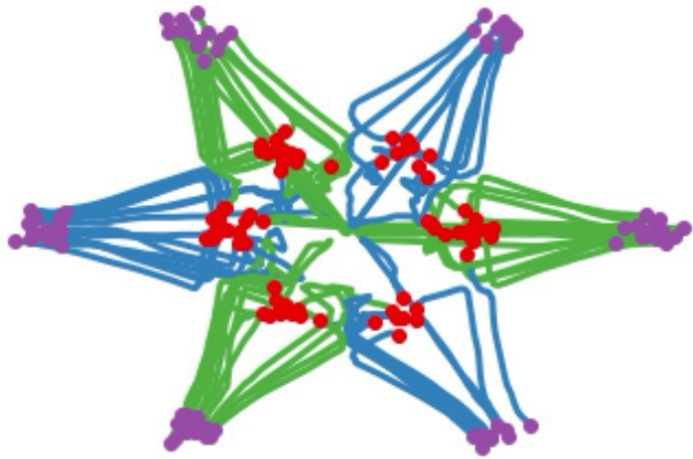
(a) The 1st rectified flow Z^1



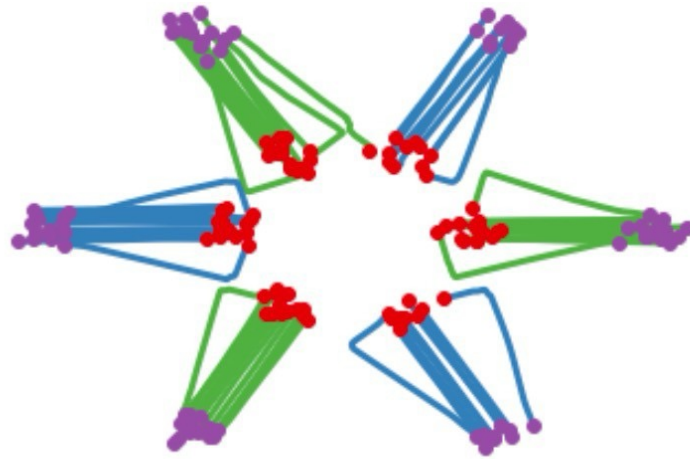
(b) Reflow Z^2

Reflow

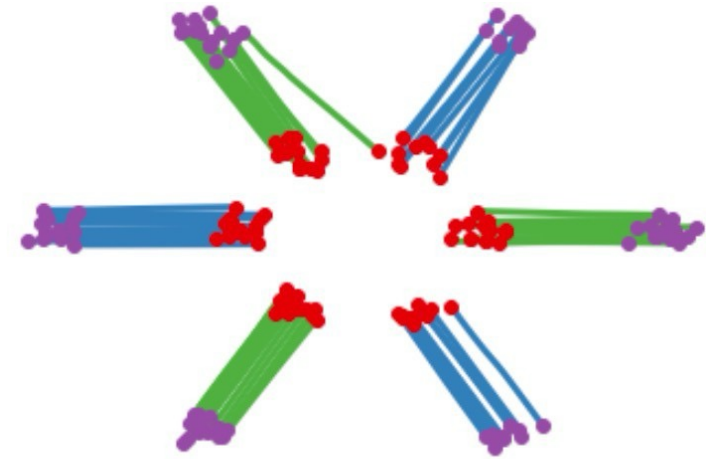
Repeat one more time. **Even straighter** trajectories.



(a) The 1st rectified flow Z^1



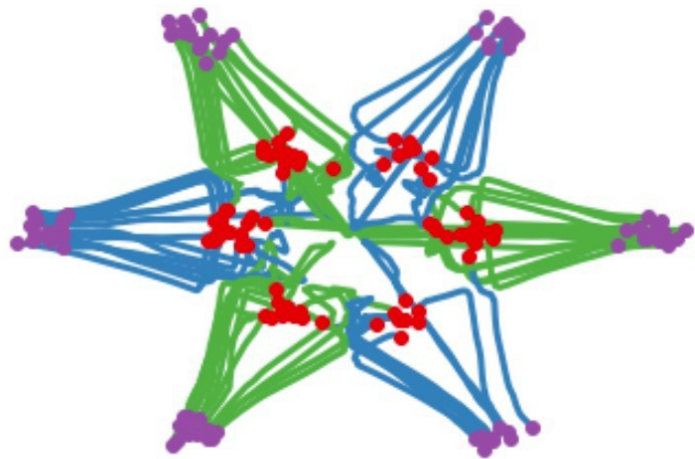
(b) Reflow Z^2



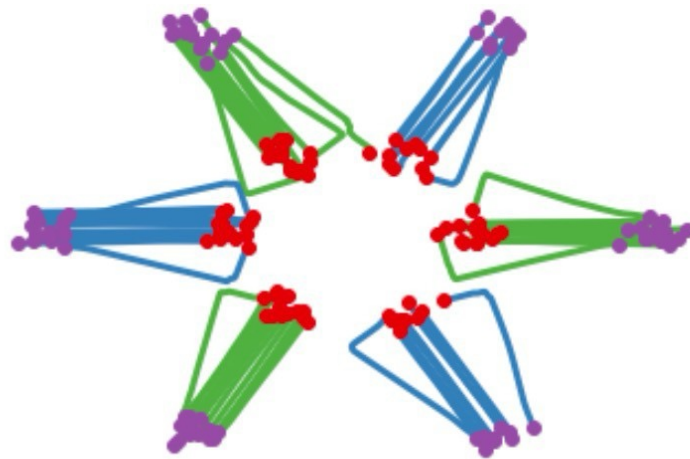
(c) Reflow Z^3

Reflow

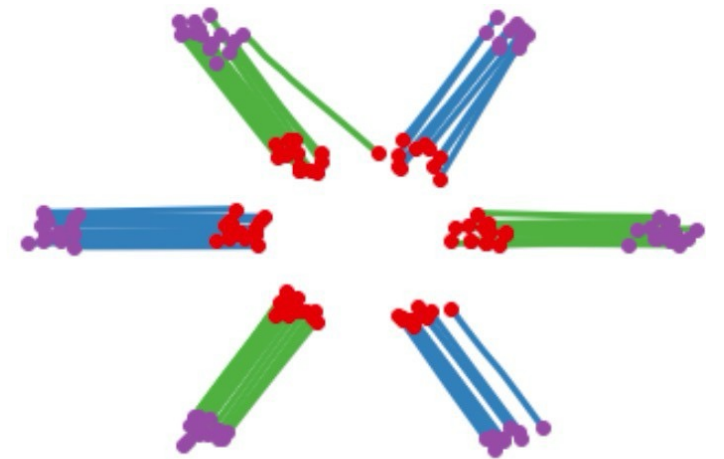
Repeat one more time. **Even straighter** trajectories.



(a) The 1st rectified flow Z^1



(b) Reflow Z^2



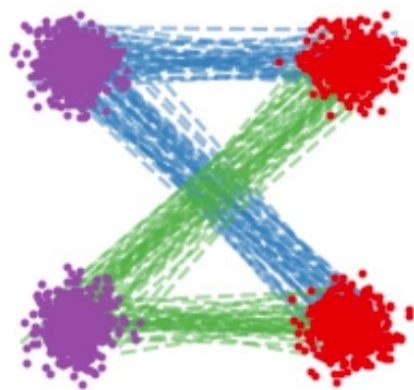
(c) Reflow Z^3

Q. 為什麼每次迭代 reflow 都會讓軌跡更直？

Q. 為什麼每次迭代 reflow 都會讓軌跡更直？

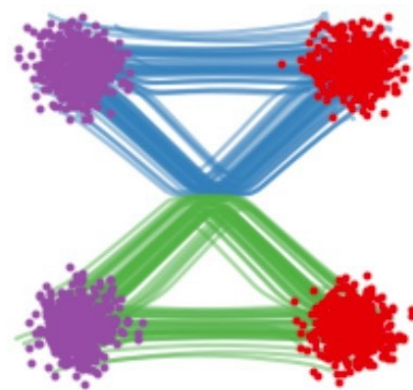
第一次訓練 (Z^1)

- 訓練資料：Randomly sampled pairs (x_0, x_1) ，其中 x_0 是 noise， x_1 是真實資料
- 問題：這些 pair 是**隨機**的，不一定是最佳的 connection
- 結果：學到的軌跡會有些彎曲，因為要適應這些不自然的配對



(a) Linear interpolation

$$X_t = tX_1 + (1 - t)X_0$$



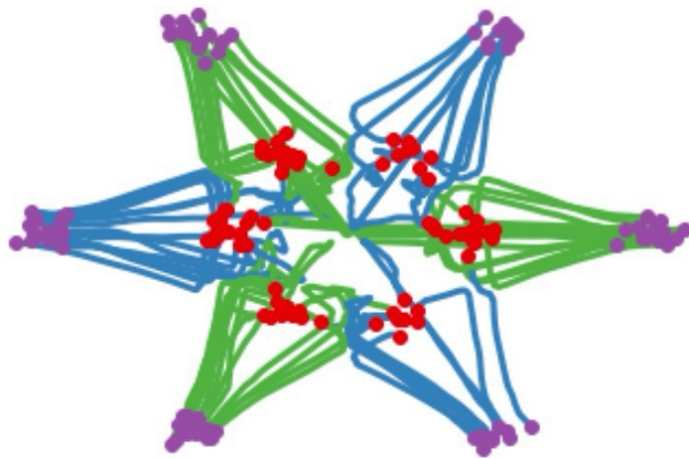
(b) Rectified flow Z_t

induced by (X_0, X_1)

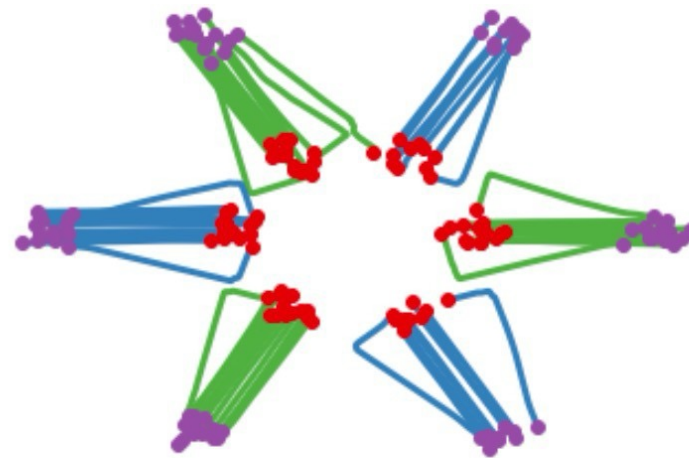
Q. 為什麼每次迭代 reflow 都會讓軌跡更直？

第一次 Reflow (Z^2)

1. 用訓練好的 v^1 從 x_0 出發，求解 ODE 得到新終點 x'_1
2. 新的訓練資料變成 (x_0, x'_1)
3. 重要：這個新配對已經沿著 v^1 的 **flow fields** 演化過了，考慮了全局結構



(a) The 1st rectified flow Z^1



(b) Reflow Z^2

Reflow

- Why? The loss

$$\mathcal{L} = \mathbb{E}_{t \sim \mathcal{U}[0,1]} \mathbb{E}_{\mathbf{x}_1 \sim p_1, \mathbf{x}_0 \sim p_0} [\|v_\theta(\mathbf{x}_t, t) - (\mathbf{x}_1 - \mathbf{x}_0)\|^2]$$

這個 loss 在懲罰「偏離直線」的 velocity。如果從 $\mathbf{x}_0$ 到 $\mathbf{x}_1$ 的最優路徑本來就很直，模型就容易學到小的 loss measure the **straightness** of the trajectories.

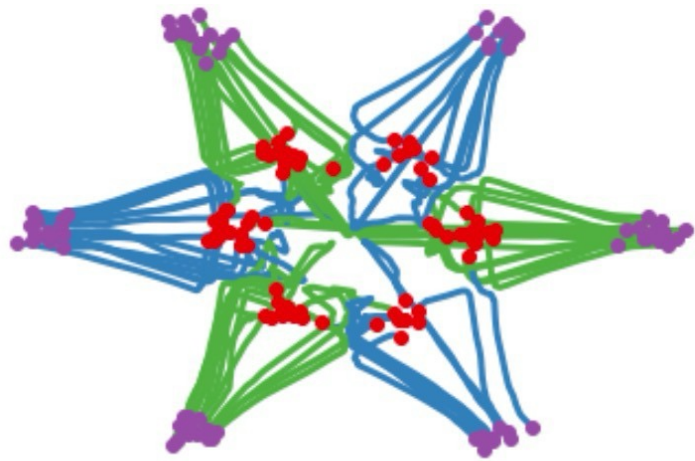
- As the flow is rectified more, the trajectories tend to become **straighter**.

Reflow 的效果：

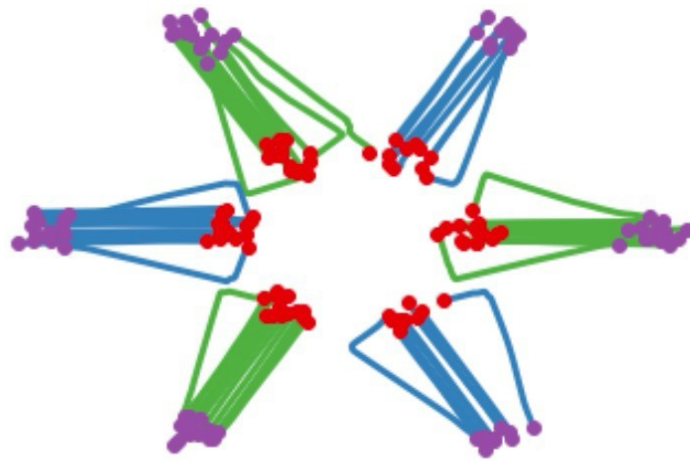
- 原始配對 $(\mathbf{x}_0, \mathbf{x}_1)$ ：可能需要繞路才能從 $\mathbf{x}_0$ 到達 $\mathbf{x}_1$
- Reflow 後 $(\mathbf{x}_0, \mathbf{x}'_1)$ ： $\mathbf{x}'_1$ 是從 $\mathbf{x}_0$ 自然流動到的點，它們之間的連接已經被「矯正」過了

Reflow

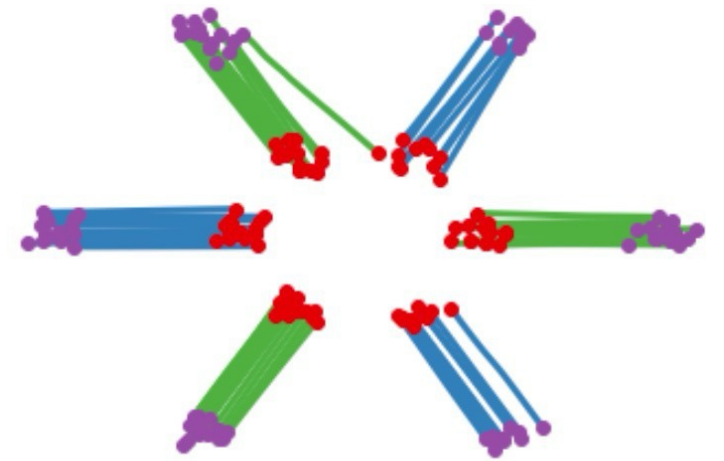
Repeat one more time. **Even straighter** trajectories.



(a) The 1st rectified flow Z^1



(b) Reflow Z^2



(c) Reflow Z^3

經過 3 次 reflow 後，軌跡已經接近完美的直線
這就是在逐步逼近**最優傳輸 (Optimal Transport)** 的解

Rectified Flow

Straighter trajectories help achieve good quality in generation, even with **one step** of the ODE solver!

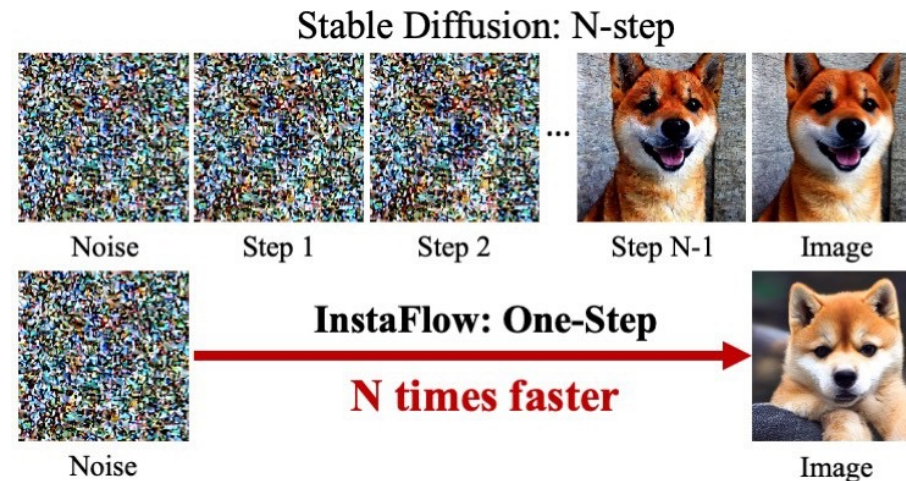
Method	NFE(↓)	IS (↑)	FID (↓)	Recall (↑)
<i>ODE</i>	<i>One-Step Generation (Euler solver, $N=1$)</i>			
1-Rectified Flow (+ <i>Distill</i>)	1	1.13 (9.08)	378 (6.18)	0.0 (0.45)
2-Rectified Flow (+ <i>Distill</i>)	1	8.08 (9.01)	12.21 (4.85)	0.34 (0.50)
3-Rectified Flow (+ <i>Distill</i>)	1	8.47 (8.79)	8.15 (5.21)	0.41 (0.51)
VP ODE [73] (+ <i>Distill</i>)	1	1.20 (8.73)	451 (16.23)	0.0 (0.29)
sub-VP ODE [73] (+ <i>Distill</i>)	1	1.21 (8.80)	451 (14.32)	0.0 (0.35)

Q. What does running one step of the ODE solver mean?

InstaFlow

Reflow Stable Diffusion!

- Note that **any generative model** can be used for reflow!
- Sample x_0 and x_1 using a Stable Diffusion for training a vector field.



InstaFlow

